SKALA

생성형AI

LLM과 Transformer 아키텍처

Day1. Transformer



2026.07

전임교수 : Byung-sun, Park

Ver.4.0

본 문서는 SK(주)AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



1. Intro : Software 3.0
2. NLP Technical Summary
3. Math Essentials for Transformer
4. Transformer

LLM과 Transformer 아키텍처

Intro : Software 3.0

- 소프트웨어 개발 방식의 진화: Software 3.0
- AI 시대, 선구자들의 시선

본 문서는 SK㈜AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

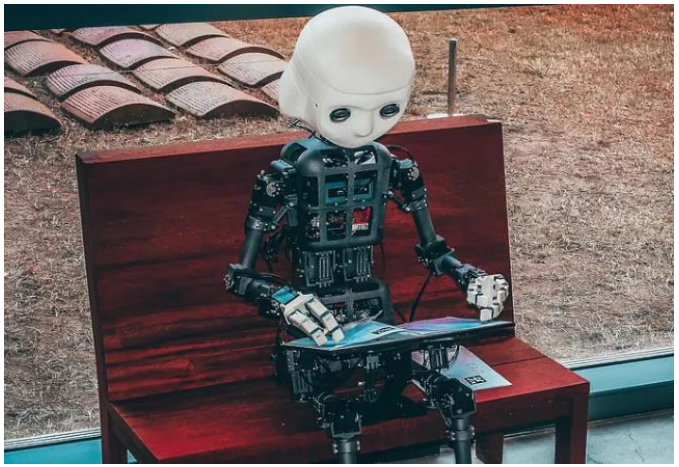
SK Inc. AX

우리는 어떤 Software 시대에 살고 있는가?

Software 3.0, 세번째 전환점에 서있는 Software

> *우리는 소프트웨어를 만들고 있는가?*
>
> *아니면 '지능이 일하는 시스템 ' 을 설계하고 있는가?*

- 지금은 코드를 작성하는 시대에서, 지능을 설계하는 시대로 전환 중..
- 규칙을 코드로 작성하는 시대나 데이터로 모델을 학습하는 시대로 진화했지만..
- 복잡한 사고와 판단은 어떻게 구현할 것인가?



Source : https://medium.com/@itamar_f/software-3-0-the-era-of-intelligent-software-development-acd3cafe6cd7

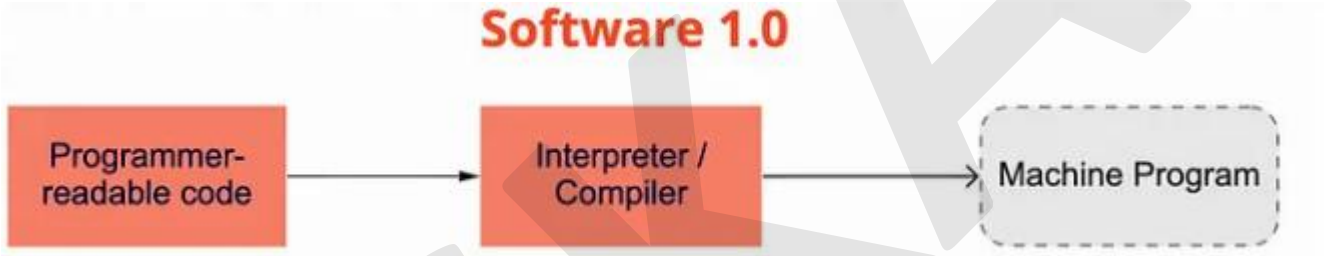
SKALA

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

Software 1.0 : 사람이 규칙을 코드로 직접 작성

Concept

- 프로그래머가 Python, C++, JS, CSS 등의 언어로 작성
- 컴파일러/인터프리터에 대한 명시적(선언적 또는 명령적) 명령으로 구성
- 프로그래머는 각 코드/명령어 줄을 작성함으로써 프로그램 내에서 원하는 동작을 수행
- 인간이 만든 소스 코드(예: 일부 .cpp 파일)가 가치 있는 작업을 수행하는 바이너리로 컴파일



Source : https://medium.com/@itamar_f/software-3-0-the-era-of-intelligent-software-development-acd3cafe6cd7

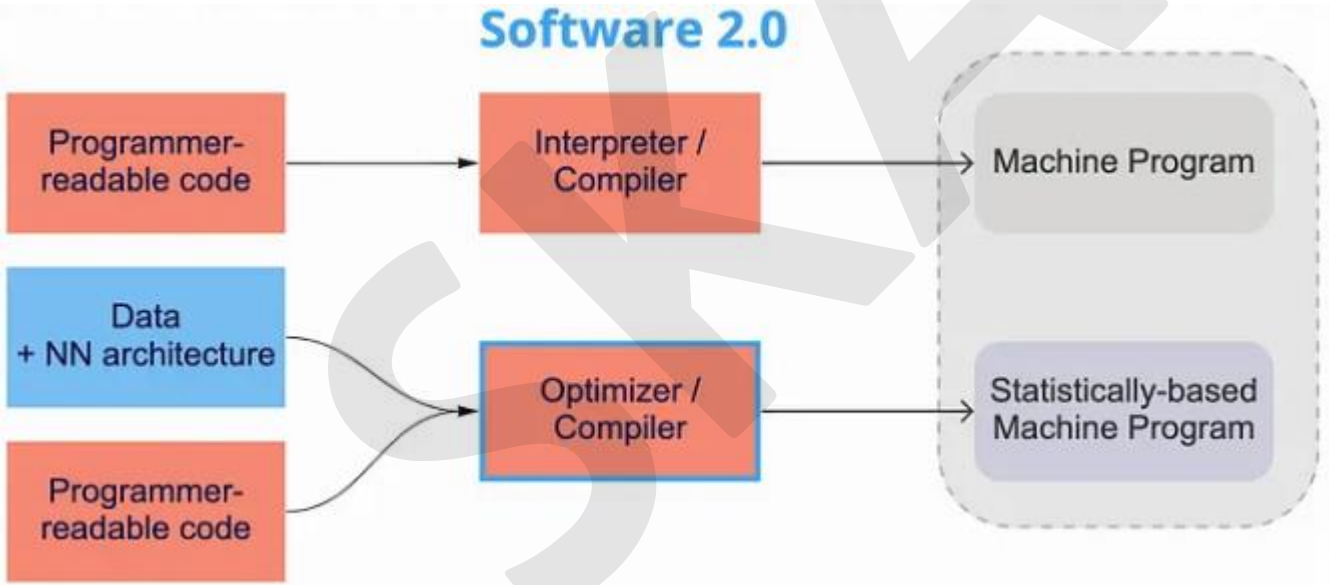
SKALA

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

Software 2.0 : 사람이 데이터를 주고 모델을 학습

Concept

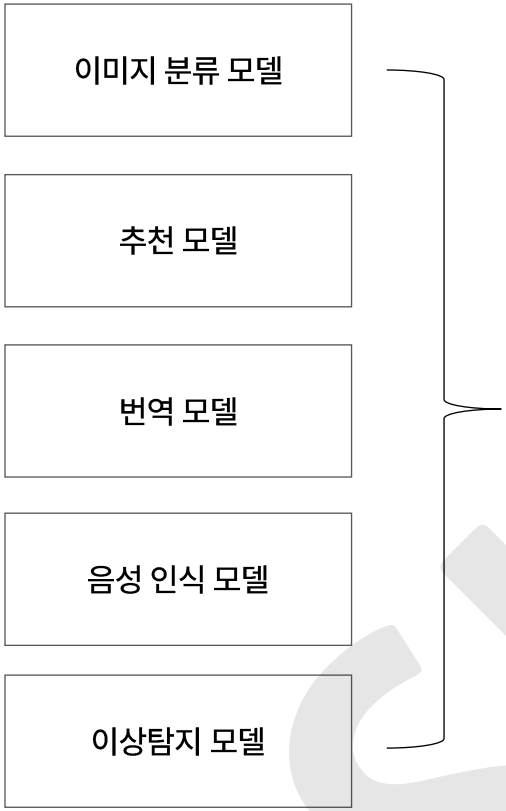
- 신경망의 가중치처럼 훨씬 더 추상적이고 인간 친화적이지 않은 언어로 작성
- 일반적인 신경망은 수백만 개의 가중치를 가질 수 있음
- 가중치가 매우 많기 때문에 이 코드 작성에는 인간이 관여하지 않음
- 소스 코드는 대부분 1) 비즈니스 로직, 데이터 전처리 등의 S/W 1.0 코드, 2) 데이터와의 상호작용, 3) 신경망(NN) 아키텍처, 4) 신경망을 컴파일하기 위한 코드로 구성



Source : https://medium.com/@itamar_f/software-3-0-the-era-of-intelligent-software-development-acd3cafe6cd7

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

Software 2.0 의 한계



Software 2.0은

모든 문제마다 모델을 새로 만들어야 함

문제(Task)마다 AI모델이 하나씩 필요..

Task-specific AI

- 문제(Task)마다 AI 모델을 새로 개발
- 데이터 수집부터 모델 설계, 학습(Training), 배포까지 반복 수행
- 모델 간 재사용이 어려움
- AI 하나 = 문제 하나

SKALA

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

Transformer와 LLM의 등장

Transformer의 등장	Foundation Model 시대의 시작	LLM의 대중화
• 2017, Attention Is All You Need 발표 • Self-Attention 구조 제안 • 다양한 언어 Task를 하나의 구조로 해결 • 범용 언어모델(Foundation Model)의 기술적 기반	• 데이터를 활용해 대규모 사전학습 • Task마다 모델을 새로 개발하지 않고 하나의 모델을 다양한 업무에 활용 • Fine-tuning, Prompting을 통해 새로운 문제에 빠르게 적용	• ChatGPT를 계기로 LLM은 누구나 사용하는 기술로 확산 • 자연어(Prompt)만으로 다양한 업무 수행 • Prompt Engineering, AI Agent 시대로 발전

✓ Task-specific AI → General-purpose AI로 패러다임 전환

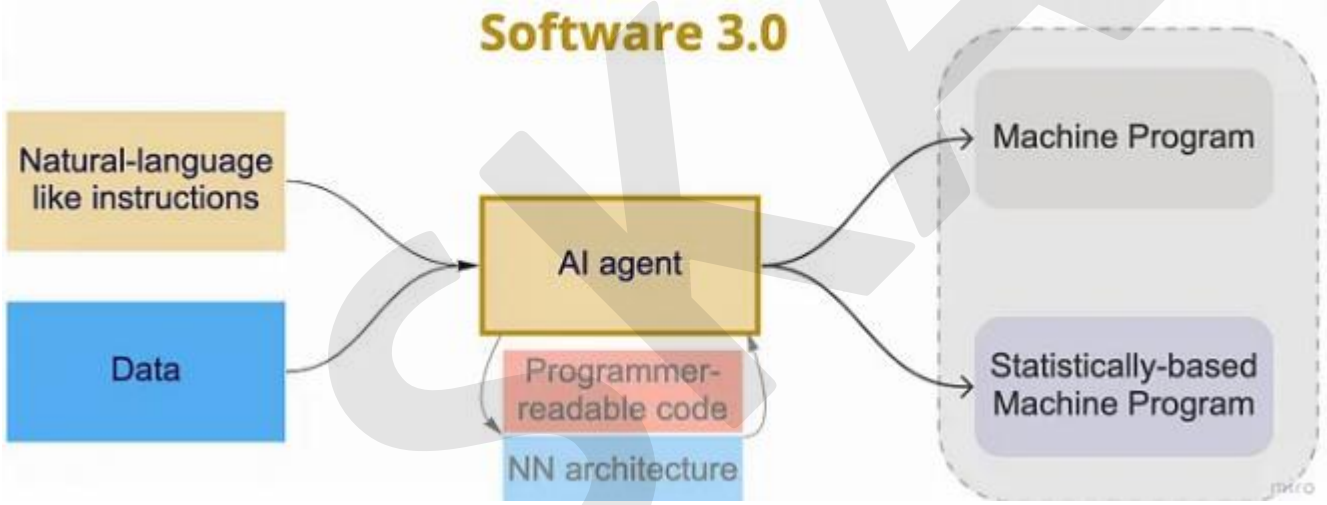
SKALA

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

Software 3.0 : 사람이 목표(역할)을 정의하면 모델이 스스로 사고하고 행동

Concept

- 소프트웨어 3.0에서는 프로그래머가 프로그램의 원하는 동작을 정의하는 일련의 명령어와 데이터셋을 제공
- 그러면 AI agent가 이러한 명령어와 데이터 세트를 기반으로 프로그램을 생성
- 이 생성 과정에는 AI agent가 코드를 생성하고 신경망 모델을 학습하는 과정이 포함될 수 있음
- 비즈니스 로직 등 필요한 부분을 AI agent에 의해 부분적으로 또는 완전히 생성



Source : https://medium.com/@itamar_f/software-3-0-the-era-of-intelligent-software-development-acd3cafe6cd7

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

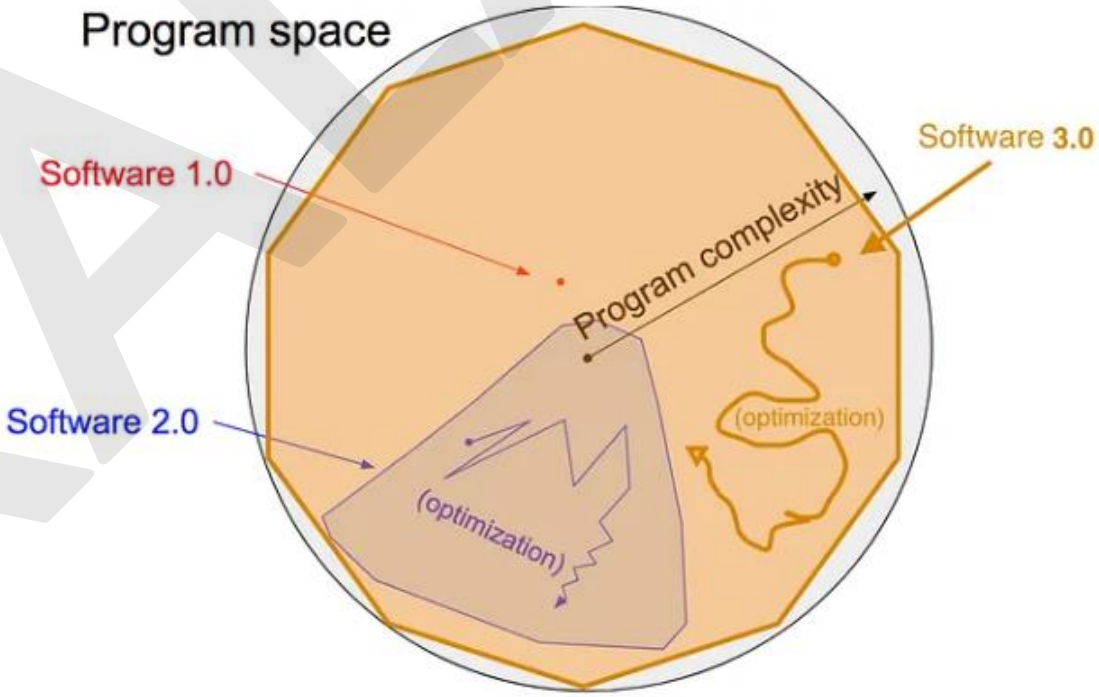
Software 3.0, 표현력의 확장 그 끝은 어디인가?

Program space

- 세상에 존재할 수 있는 모든 프로그램의 집합
- SW 3.0의 표현력(프로그램 공간)은 SW 1.0과 SW 2.0 프로그램을 포함

Software 3.0

- 프로그램을 더 잘 만드는 기술이 아니라, 하나의 거대한 모델이 대부분의 프로그램 공간(Program Space)을 포괄하도록 만든 패러다임의 변화
- 이미 존재하는 거대한 Program Space에서 Prompt를 통해 원하는 프로그램을 즉시 끌어내는 방식으로 진화



Source : https://medium.com/@itamar_f/software-3-0-the-era-of-intelligent-software-development-acd3cafe6cd7

SKALA

Software, 이렇게 바뀌어 왔다 (개발 방식의 진화)

Software 3.0, 패러다임의 전환

Software 1.0

- 전체 애플리케이션의 상당 부분을 **프로그래머가 수동으로 작성**
- 대부분의 논리와 코드는 최적화 프로세스 중에 기계에 의해 제안되지 않음

Software 2.0

- 대부분의 **머신 러닝 기반 실무 애플리케이션**에서 표준화된 신경망 아키텍처와 학습 시스템을 활용 검토
- 특정 문제에 알맞은 알고리즘의 선정과 레이블이 지정된 **데이터셋을 큐레이팅, 확장, 수정, 정제하는 일이 중요**

Software 3.0

- 최신 대규모 언어 모델(LLM) 기술과 학습 체계 덕분에 프로그래밍 패러다임에 새로운 변화가 일어나고 있음
- **LLM을 활용한 지능형 에이전트가 코드를 작성(제안)**하는 방식

SKALA

Software 3.0 : 언어가 코드가 되는 순간

자연어가 새로운 프로그래밍 언어가 된다

Traditional S/W	Software 3.0 (Now)
• 코드 • 함수 • 프로그램	• 자연어 • 프롬프트 • 모델 + 컨텍스트

> *이제 소프트웨어의 인터페이스는 API가 아니라 언어(Language) 다*

Key message

- 개발자는 코드 작성자에서 문제정의자 혹은 사고 설계자로 역할 변화
- 소프트웨어는 더 이상 컴파일 하지 않을런지 모른다. 대신 대화하고 지시하는 존재

SKALA

Software 3.0 시대에서의 LLM

LLM은 모델이 아니라 새로운 실행환경

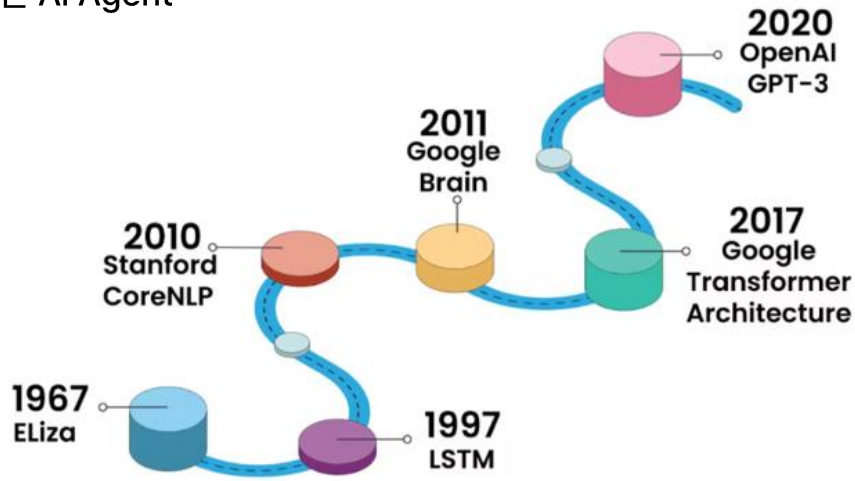
패러다임 변화 속, LLM의 역할

- LLM은 단순한 AI가 아니라 사고를 실행하는 플랫폼
- 추론엔진, 지식 인터페이스, 자연어 컴파일러, 에이전트의 두뇌..

LLM의 원리와 한계를 잘 이해한다는 것은?

- LLM을 잘 쓰는 사람을 넘어서 LLM으로 시스템을 만드는 사람으로..
- LLM 기반의 다양한 추론 시스템 설계를 가능하게 함
- Ex) 반복 업무를 수행하는 AI Agent, 복잡도가 높고 불안정한 요구사항을 처리하는 Ai Agent

> *LLM은 도구가 아니라 함께 일하는 새로운 형태의 소프트웨어다*



SKALA

Limitation of Software 3.0

Software 3.0은 인간과 기계가 긴밀히 협력하여 지능형 소프트웨어 개발을 구현하는 것

- **Black Box**
  - Software 2.0의 한계가 그대로 이어짐
  - 수십억 개의 파라미터가 어떻게 특정 답을 만들었는지 설명하기 어려움
  - 잘못된 결과가 나왔을 때 원인을 추적하기 쉽지 않음
- **Bias**
  - LLM은 웹수집 데이터를 바탕으로 데이터 편향, 문화적 편향, 오래된 정보, 잘못된 정보까지 함께 학습
  - Prompt를 잘 작성해도 근본적으로 학습된 편향은 완전히 제거되지 않음, silently fail
- **Hallucination**
  - LLM은 정답을 찾는 모델이 아니라 다음 토큰을 가장 그럴듯하게 예측하는 모델
  - 그래서 존재하지 않는 논문, 잘못된 수치, 가짜 출처, 틀린 코드를 매우 자연스럽게 생성할 수 있음
- **Non-deterministic**
  - 같은 Prompt, 동일한 입력에도 조금씩 다른 결과 → 항상 동일한 프로그램처럼 동작하지 않음

SKALA

Software 3.0 을 향하여..

Software 3.0은 인간과 기계가 긴밀히 협력하여 지능형 소프트웨어 개발을 구현하는 것

Benefit

- 결정론적 모델과 통계적 모델의 조합을 제공 : 결정론적 업무는 기존 코드가 수행하고, 판단·추론·생성은 LLM 이 수행하는..
- 코드를 수정하거나 변경하는 것이 더 쉬움 (vs 프로그래머가 신경망 가중치를 조작하는 것)
- 더 많은 사람들이 프로그래머가 될 수 있음 : Application의 많은 부분이 자연어와 같은 명령어로 표현되므로 프로그래머 풀을 확대하는 것이 더욱 용이
- SW 1.0과 SW 2.0 스택 모두 온라인 교육 자료의 증가에도 불구하고 상당한 진입 장벽이 있는데, 이의 해소에 기여
- 개발 속도 향상 : SW 애플리케이션 구축에 자동화가 도입되므로 SW 프로그램을 더 빠르게 작성할 수 있음
- AGI(Artificial General Intelligence)를 향한 교두보

SKALA

새로운 시대, AI 선구자들의 시선

자신이 개척한 AI 기술의 위험성에 대한 AI 대부의 경고



Source : https://www.wsj.com/tech/ai/a-godfather-of-ai-just-won-a-nobel-he-has-been-warning-the-machines-could-take-over-the-world-b127da71?utm_source=chatgpt.com

새로운 시대, AI 선구자들의 시선

위험할 정도로 강력한 AI가 멀지 않았다



새로운 시대, AI 선구자들의 시선

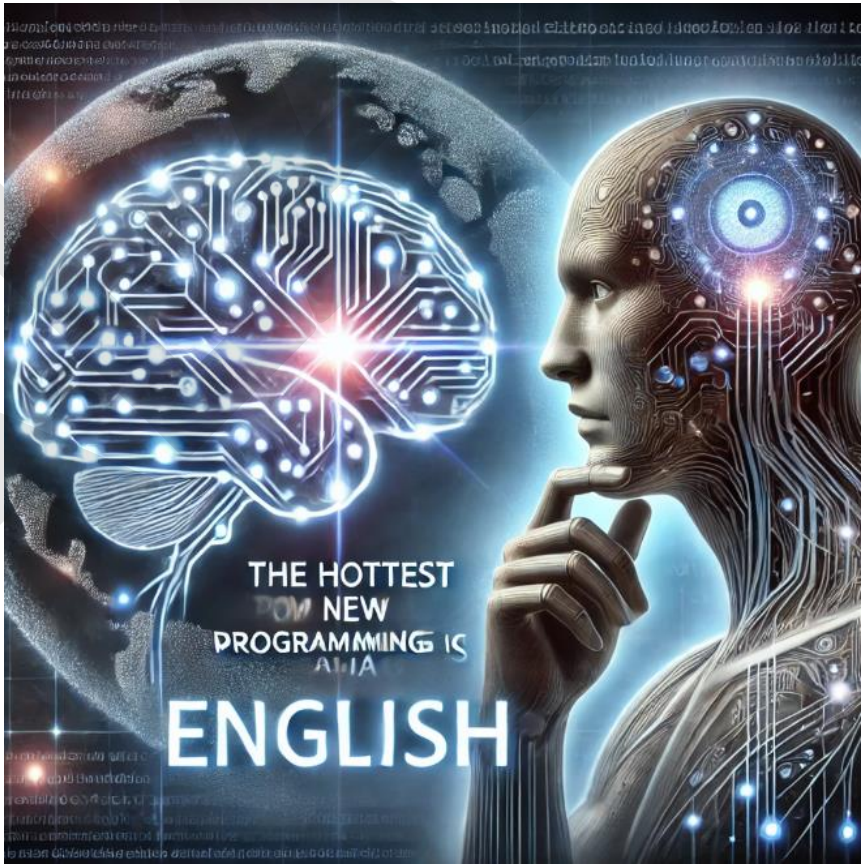
AI, 우리 곁의 일상이 되다



새로운 시대, AI 선구자들의 시선

자연어로 AI를 프로그래밍하는 시대





SKAI

새로운 AI 시대의 도래

Large Language Models (LLMs)

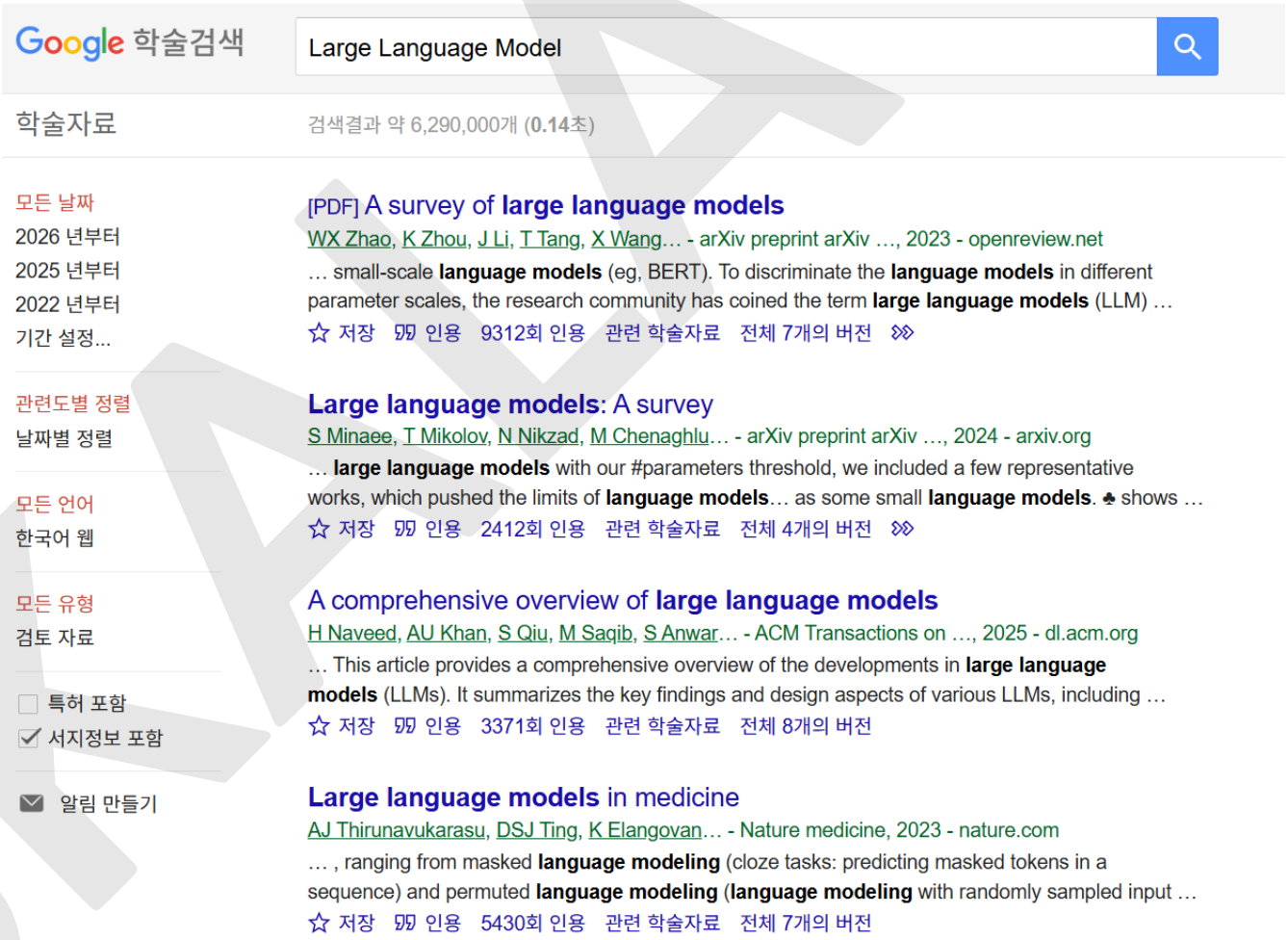
“Large Language Models (LLMs) are advanced AI systems trained on vast amounts of text data to understand, generate, and process human language, enabling them to perform tasks like translation, summarization, and question answering with remarkable accuracy.”

Brown et al., 2020, “Language Models are Few-Shot Learners”, NeurIPS

새로운 AI 시대의 도래

Large Language Models (LLMs)

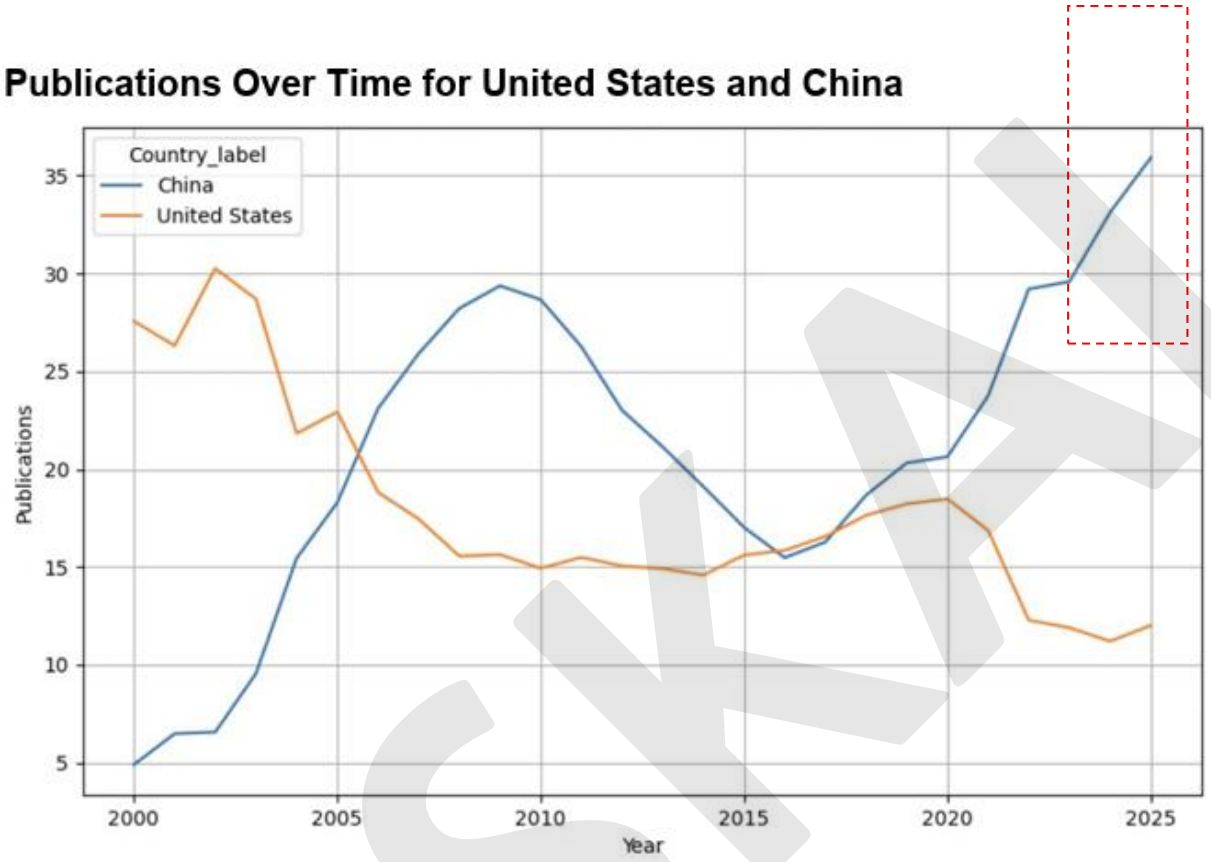
- ✓ AI 연구와 서비스 발전 속도가 매우 빨라지는 추세
- ✓ Google Scholar 기준 “Large Language Model” 논문 검색 결과 620만편+



SKALA

새로운 AI 시대의 도래

Leading Countries in AI Publications, 2000-2025



- 중국은 또 한번의 연구적 성과의 약진을 통해 논문의 출판량에서 미국을 압도
- 그 이면에서 생성형AI는 어떤 역할을 했을까?

SKALA

새로운 AI 시대의 도래

AI, 2~3년 주기로 Quantum Jump

- 2012 제프리 힌튼 토론토대 교수팀이 이끈 '알렉스넷(AlexNet)'이 이미지 인식 대회 이미지넷에서 우승
- 2014 딥마인드가 구글에 인수되며 알파고 개발 본격화
- 2016 이세돌 9단과 알파고의 대국이벤트, 이른바 '알파고 쇼크'가 전 세계를 강타
- 2017 AI의 역사를 바꾼 논문 'Attention Is All You Need'가 발표, by Google
- 2020 트랜스포머 아키텍처는 오픈AI의 대규모 언어 모델(LLM) 개발을 촉발, 오픈AI가 GPT-3 발표
- 2022 거대 파라미터를 가진 AI 모델 개발 경쟁시대, ChatGPT가 세상에 등장
- 2026 ?

SKALA

증강지능, Augmented Intelligence

AI, 적이 아닌 친구로..

Concept

- 증강 지능(Augmented Intelligence)은 AI를 인간을 대체하는 기술이 아니라, 인간의 능력을 확장하고 보완하는 기술로 보는 개념
- AI가 인간의 지식, 경험, 판단, 의사결정을 지원하여 더 나은 성과를 내도록 하는 협업 중심의 AI 활용 방식
- AI는 인간의 경쟁자(Competitor)가 아니라 협업 도구(Co-pilot)
- 목표는 인간을 대체하는 것이 아니라 인간의 생산성과 의사결정 품질을 향상시키는 것

Use-case

- Retail에서 AI는 유동 인구, 구매 이력, 상품 판매 데이터를 분석하여 최적의 매장 레이아웃과 상품 배치를 제안
- 상품 진열과 판매 전략은 상품 담당자가 최종 결정

✓ 우리가 본질적으로 대체되는 미래를 원하지 않는다면..

✓ 인간이 생산적인 역할을 수행하는 미래를 원한다면.. 증강 지능의 중요성은 점점 더 커질 것

SKALA

LLM과 Transformer 아키텍처

NLP Technical Summary

- 빠르게 살펴보는 NLP의 어제와 오늘 (NLP에서 LLM까지)

본 문서는 SK㈜AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

언어 이해의 시작, NLP(Natural Language Processing)

사람이 사용하는 언어를 Machine이 어떻게 이해할 수 있을까?



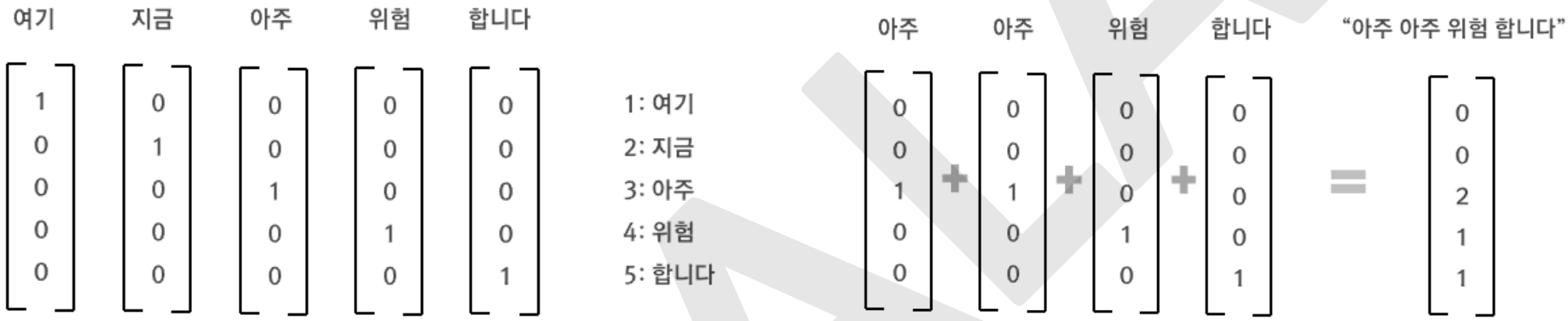
SKALA

Machine이 이해할 수 있는 언어의 형식

- 언어를 Machine이 이해하려면 수치로 변환해야 함
- 어떻게 수치로 변환할 수 있을까?
- 수집하여 가지고 있는 모든 단어들..Corpus
- NLP Task를 위해 Corpus로부터 뽑은 단어들..Vocabulary

언어 이해의 시작, NLP(Natural Language Processing)

사람이 사용하는 언어를 Machine이 어떻게 이해할 수 있을까?



Bit를 활용해 연속형 수치(Vector)로 변환

- 모든 단어에 Indexing 하는 것과 유사
- 0/1 이진법을 활용해서 "N(단어개수) * 1"의 column vector으로 표현하면 그게 바로 "One-hot vector"

단어의 빈도를 활용한 문장 표현

- 문장을 단어의 빈도수로 표현한 것이 BoW(Bag of Words) vector
- BoW vector 의 size는?
- 각 단어가 몇 번 사용되었는지 알 수 있지만, 사용된 단어의 순서는 알 수 없음



This image was generated by G.AI

SKALA

빈도기반의 표현법, 조금 더 신경 써보자

사람이 사용하는 언어를 Machine이 어떻게 이해할 수 있을까?

N-Gram : 순서를 고려해 보자

- 연속된 n개의 단어 뭉치
- 순서정보를 일부 보존하여 국소적 의미를 구분 할 수 있음
- Ex) (not good) <> (good not)
- bi-gram (n=2)을 만든다면 데이터 속 순차적인 모든 단어 pair쌍을 vocabulary에 넣어야 함
- n이 커질수록 너무 많은 단어 쌍이 생김, 현실적으로 확장에 제한

I am studying bigram model

⇩

{ "I am", "am studying", "studying bigram", "bigram model" }

TF-IDF : 어떤 단어가 중요한 단어일까?

- Term Frequency - Inverse Document Frequency
- Term Frequency : 많이 출현하는 단어가 중요
- Inverse Document Frequency : 다른 문서에도 자주 출현하는 단어는 중요도가 떨어짐

$$TF(t,d) = \frac{(Number\ of\ occurrences\ of\ term\ t\ in\ document\ d)}{(Total\ number\ of\ terms\ in\ the\ document\ d)}$$

$$IDF(t,D) = \log_e \frac{(Total\ number\ of\ documents\ in\ the\ corpus)}{(Number\ of\ documents\ with\ term\ t\ in\ them)}$$

$$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$

SKALA

빈도기반 표현의 한계는 무엇일까?

BoW, TF-IDF Vector 등 빈도 기반 언어 표현의 한계

- ✓ 순서가 중요하지 않은 Task (Topic Classification, 문서검색) 에서는 강점을 가지지만..
- ✓ 단어 나열의 순서가 중요한 Task라면 활용이 어려움
- ✓ 사용되는 단어수(Corpus)가 커질수록 엄청나게 커지는 vector size.. 어떤 문제를 가져올까?.. 차원의 저주(Curse of Dimensionality)
- ✓ 단어 간의 관계 표현에 취약, 즉 의미를 알 수 없음

"사과는 가방보다 오렌지와 관련이 있음"



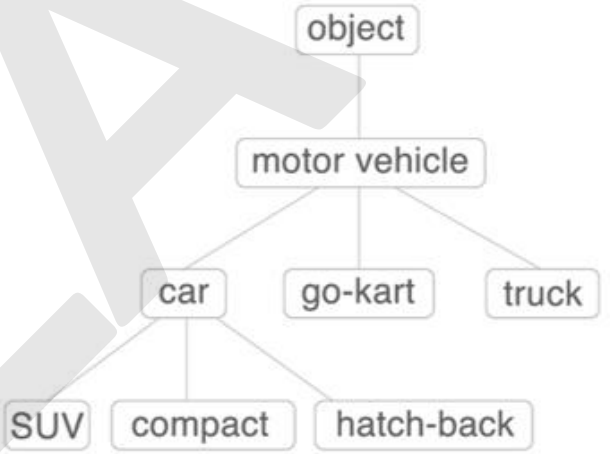
"동생"과 "아우"는 쓰임이 다르지만 의미가 같음



SKALA

모든 단어들의 관계를 사람이 직접 정의할 수 있을까?

단어 간 의미 관계를 직접 구축하다, 의미 관계를 담고 있는 Thesaurus의 구축 시도



Thesaurus란?

- 단어의 의미를 정의하여 저장하고 있는 사전
- 기본적으로 동의어 또는 유의어가 한 그룹으로 분류되어 있음
- 단어 간 다양한 관계를 그래프 구조로 정의함

WordNet

- 프린스턴 대학교에서 1985년부터 구축하기 시작한 시소러스
- 표제어를 통해 특정 단어가 갖는 다양한 의미 표현 (하나의 의미 그룹에 여러 단어가 포함)
- Synset(동의어 집합), Hypernym(상위어), Hyponym(하위어) Meronym(부분 관계) 등 을 그래프로 연결
- 단어 네트워크를 사용하여 유의어를 찾거나 단어 간 유사도를 구할 수 있음

```
print(car.path_similarity(novel))
print(car.path_similarity(dog))
print(car.path_similarity(motorcycle))
```

```
0.05555555555555555
0.07692307692307693
0.3333333333333333
```

- 신조어 또는 의미가 변하는 어휘에 대응하기 어려움
- 시소러스 구축에 비용이 많이 듦

✓ 세상의 모든 단어를 사람이 직접 정의하는 것은 불가능

SKALA

맥락(Context)을 이해한다는 것

You shall know a word by the company it keeps. ,Zellig Harris

분포가설(distributional hypothesis)

- 단어의 의미는 그 단어가 나타나는 문맥에 의해 정의된다.
- 즉, 비슷한 문맥(주변 단어)에서 쓰이는 단어들은 의미도 비슷하다.
- 언어학적으로, 단어의 의미를 그 분포(distribution), 즉 주변 문맥을 기반으로 분석할 수 있음을 시사

➢ 같은 부모한테서 태어난 형제 중 나이가 적은 쪽을 부를 때 아우라 부른다

➢ 친척 형제들 혹은 같은 부모에게서 태어난 형제 중에서 나이가 적은 사람을 동생이라 부른다.

→ "아우"와 "동생 " 의 주변 단어가 유사하다면 그 의미도 유사

- *"Words that are similar in meaning occur in similar contexts."*
- *"The meaning of a word is determined by the contexts in which it occurs."*
- *"If two elements have the same distribution, we can substitute one for the other anywhere in the text."*

Zellig Harris (1954), Distributional Structure

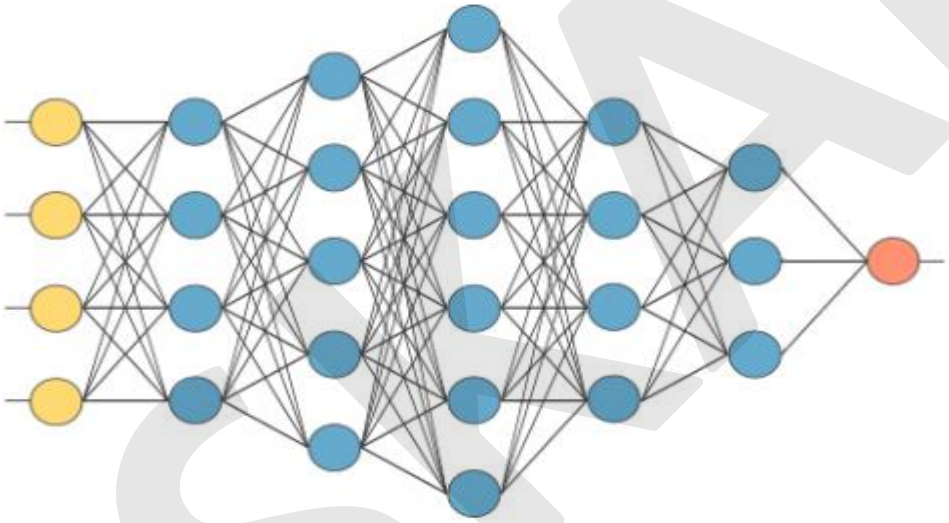
SKALA

의미적 추론을 가능하게 한 워드 임베딩(Word Embedding)

Deep Learning을 통한 분포가설의 구현, Word2Vec

빠르게 살펴보는 Deep Learning 학습 과정

- Input 데이터를 넣었을 때, 나와야 하는 답(Y)를 정해 놓고, 그렇게 되도록 중간 방정식을 변화시키는 과정
- 내가 원하는 중간 방정식을 얻어서, 그 방정식을 활용하겠다(예측/분류)
- 필요한 것은 Input-Lebel(Y)쌍의 데이터 집합과 방정식을 변화시킬 수 있는 알고리즘



SPALA

의미적 추론을 가능하게 한 워드 임베딩(Word Embedding)

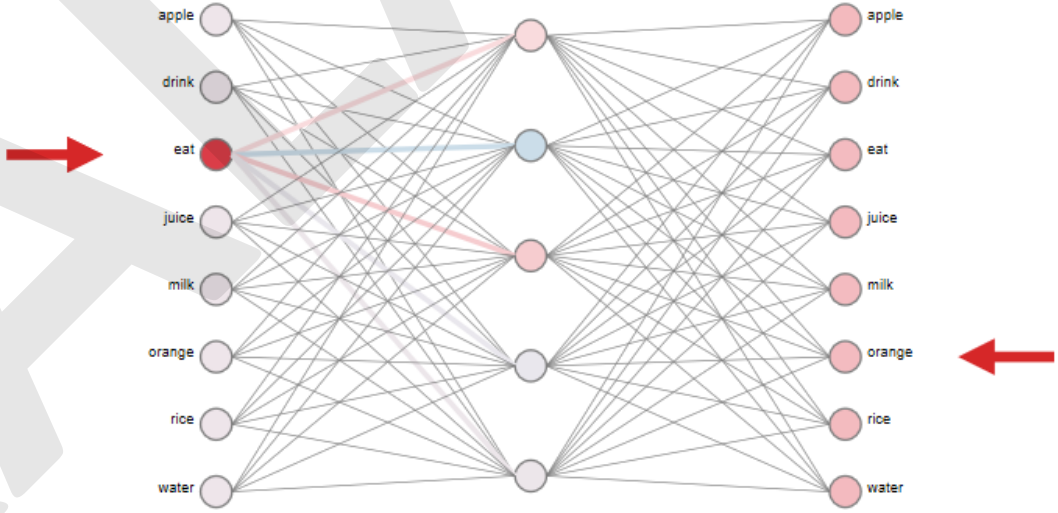
Deep Learning을 통한 분포가설의 구현, Word2Vec

Word2Vec 학습 과정

- 함께 출연한 단어로 이루어진 Train Data를 구성한 후 주변단어와 타겟 단어간의 분류(Classification) 문제로 정의한 후 학습
- Neural Network를 통해 학습이 진행될 수록 주변 단어들과의 관계가 임베딩 공간에 투영됨

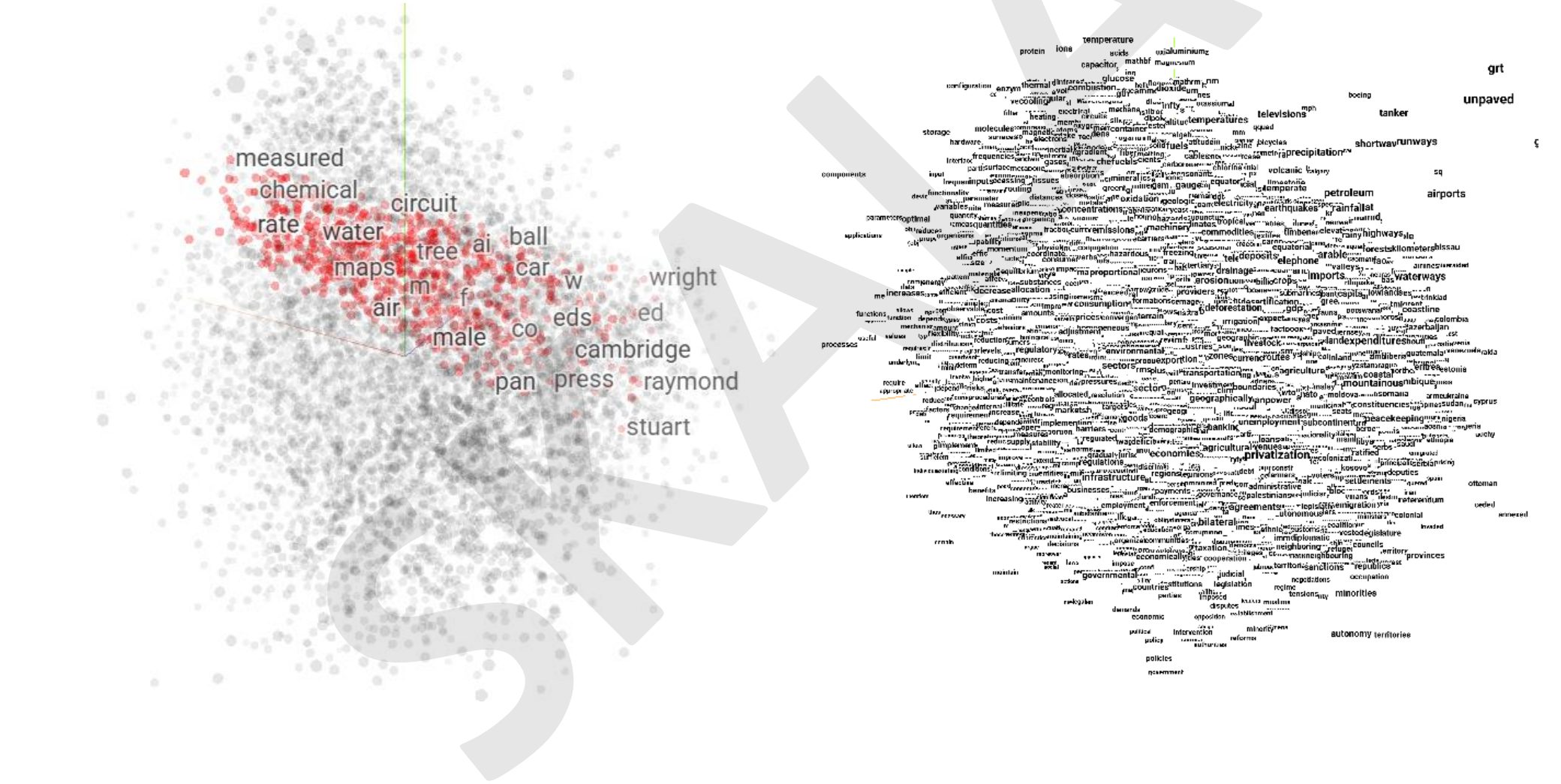


Input	Target(Y)
You	Goodbye
Say	Goodbye
And	Goodbye
I	Goodbye



https://ronxin.github.io/wevi/

[참고] Embedding Space



https://projector.tensorflow.org/

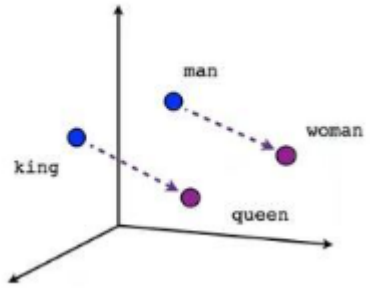
의미적 추론을 가능하게 한 워드 임베딩(Word Embedding)

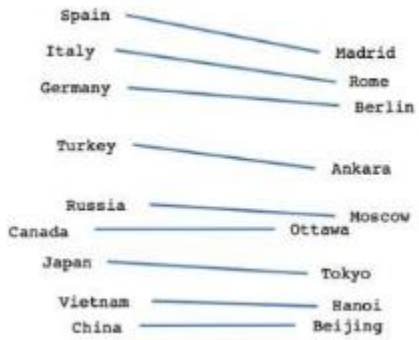
Deep Learning을 통한 분포가설의 구현, Word2Vec

Reasoning

- ‘행복’ 과 비슷한 의미의 단어들은 무엇이 있을까?
  - 기쁨, 즐거움, 행운…
- King – Man + Woman = ( )
  - 단어간의 더하기, 빼기가 가능할까?

- 충분한 양의 유사 문장을 찾아서 읽다 보면 위 질문에 답할 수 있음
- 단어간 관계는 수많은 문장 읽기를 통해 형성됨
- Word2Vec을 통해 생성된 벡터는 유사도 기반의 색인 등에 활용할 수 있음





SKALA

What is LM(Language Model)?

LM은 가장 자연스러운 단어 시퀀스를 찾아내는 모델

퇴근 후 공항에 택시를 타고 갔는데, 탑승시간에 늦어서 결국 비행기를 ( )

- 앞에 순차적으로 나열된 단어들을 살펴보았을 때 ( )에 들어갈 만한 그럴듯한(likeliness) 단어는?
- 앞의 단어들의 history를 보고, 다음 단어를 추론하는 것은 언어를 이해하는 것(Language Understanding)과 일맥상통

- 가장 자연스러운 단어 시퀀스를 찾아낸다 = 단어 시퀀스에 확률을 할당한다
- 어떤 문장(단어 시퀀스)이 주어졌을 때, "얼마나 그럴 듯 하냐"를 수리적(확률)으로 나타내거나 문장의 앞부분이 주어졌을 때 다음 단어를 예측할 수 있음

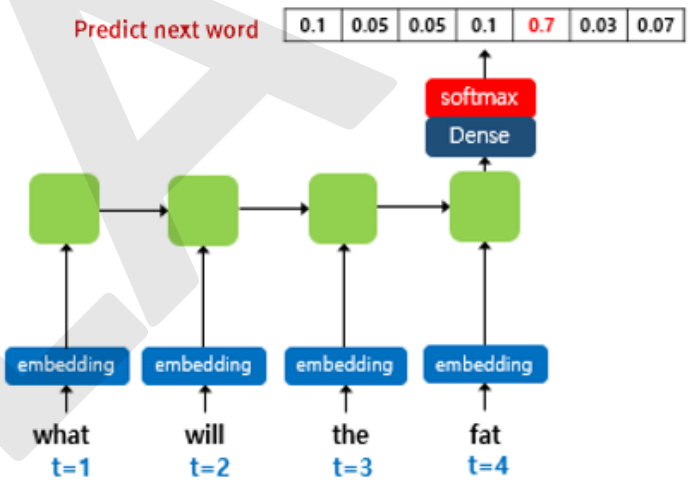
<u>다음 단어를 어떻게 예측하나?</u>

- 앞에 어떤 단어들이 나왔는지를 고려하여 후보가 될 수 있는 여러 단어들에 대해서 등장 확률을 추정하고 가장 높은 확률을 가진 단어를 선택
- 등장 확률은 가지고 있는 데이터로부터 계산할 수 있음, 이것은 마치 사람이 지속적으로 읽고 듣는 것을 통해서 언어를 배우는 것과 유사

SKALA

현재와 과거의 의존성을 모델링하는 RNN(Recurrent Neural Network)

순차적인 문장의 단어는 시계열 모델에 적합한 데이터



문장의 단어를 순차적으로 처리하는(컨베이어 벨트) RNN

- 오늘 주가를 예측한다면 어제 주가가 x원 이었다는 사실이 중요할 것
- 글을 읽고 맥락을 파악하는 행위는 한 단어, 한 단어 순차적으로 보고 이해하는 것
- RNN 은 앞 선 단어의 기록을 hidden state(h)를 통해 기억하고 있고, 이를 현재의 예측(다음 단어는 무엇?)에 활용
- hidden state(h)는 input들 간의 의존성을 저장하는 메모리

RNN의 문제점, vanishing gradient

- Neural Network의 학습 과정에서 기울기가 점점 작아져 앞 시점이 거의 학습되지 않는 현상
- 과거 정보가 현재까지 전달되지 않는 문제
- 시계열이 길수록 장기 의존성(long-term dependency) 학습 실패

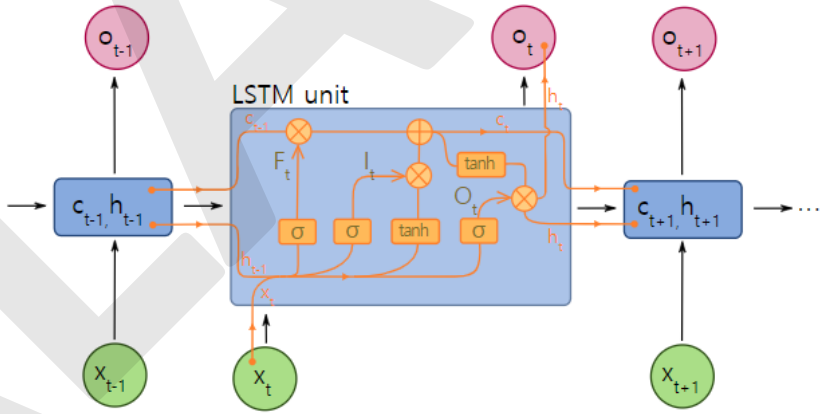
SKALA

보다 많은 맥락을 LM에 효율적으로 담아 생성하자

단어의 순차적인 컨베이어 벨트에서 문장을 압축해 한번에 넘기는 구조로..

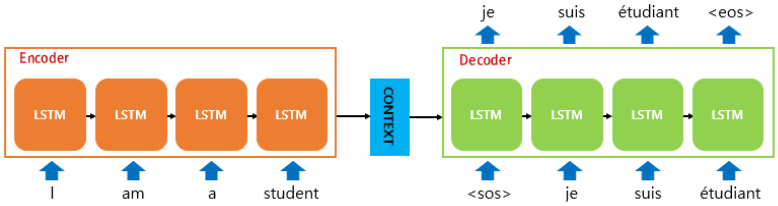
<u>장기 의존성 소실 문제를 해결하기 위한 후속 연구, Gate mechanism</u>

- LSTM(Long Short-Term Memory)
  - 모델 안에 여러 개의 gate를 추가하여 input, output, memory 간 흐르는 정보를 보다 섬세하게 제어
  - RNN보다 구조가 복잡하고 더 많은 연산비용이 필요
- GRU(Gated Recurrent Unit)
  - LSTM과 유사한 성능을 유지하면서 연산비용을 경량화



<u>전체 맥락을 하나로 압축하여 전달하자, Sequence-to-Sequence Based LM</u>

- Encoder의 전체 단어의 맥락을 가지고 있는 Hidden state vector(Context vector)를 Decoder로 전달함
- Decoder에서는 Input과 Encoder의 Context vector를 고려하여 출력
- 입력 Sequence가 길어지면, 출력 Sequence의 정확도가 떨어짐
- Why? 너무 많은 단어가 하나의 sentence vector로 압축되어 정보 소실



보다 많은 맥락을 LM에 효율적으로 담아 생성하자

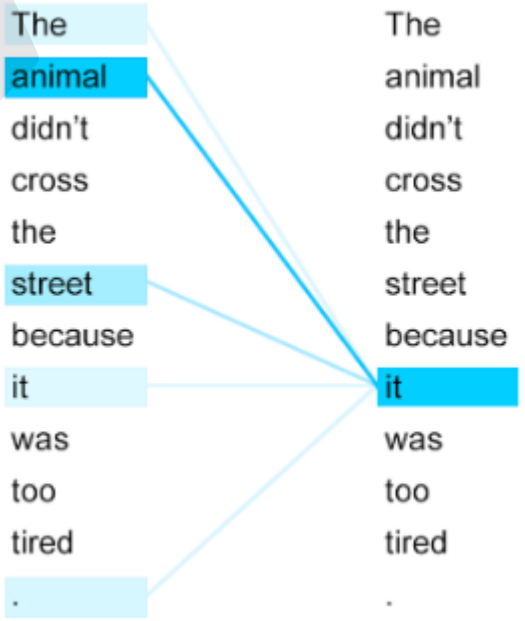
장기의존성 문제, 병렬처리 문제의 한계를 극복한 Attention 매커니즘

퇴근 후 티켓을 들고 공항에 택시를 타고 갔는데, 탑승시간에 늦어서 그것을 (환불했다)

- "그것이" 이 무엇일까?, 유추하려면 마지막 단어 (환불했다)를 고려해야 함
- 앞선 Model은 순차적으로 단어를 처리하기 때문에 뒷 단어를 고려하는 처리가 불가능함

<u>순서의 파괴, 전체를 조망하는 Attention Based LM, Transformer</u>

- Decoder에서 단어 출력시마다 매 시점 Encoder의 전체 입력 문장을 다시 보자
- 입력 Sequence를 하나로 압축한 context vector를 보지 않고, 매 시점 Encoder의 hidden state를 전부 참고하자
- 지금 출력하고 있는 단어에 따라 살펴봐야 할 단어들은 다르다.
- 각 단어가 출력할 때 도움이 되는 정도를 Scoring하여 함께 전달 (= Attention Score)
- Attention 기반의 병렬처리가 가능한 생성 모델 구조인 Transformer는 대규모 사전 학습 모델(LLM)의 시대를 염

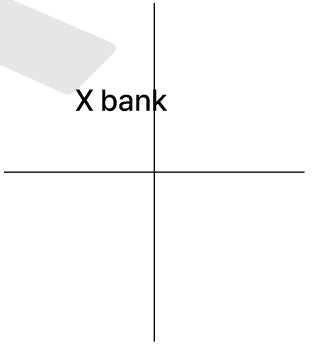


SKALA

문맥적 뉘앙스까지..Contextual Embedding

Attention 매커니즘으로 극복한 Word Embedding의 한계

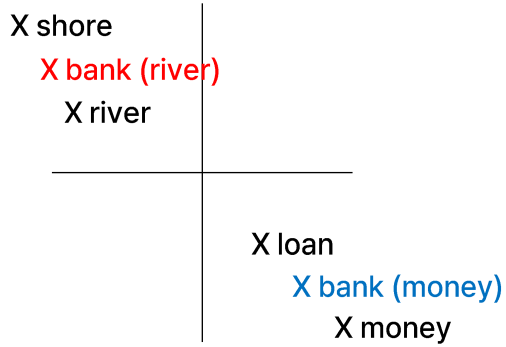
<Word Embedding>



Word Embedding의 한계

- Word2Vec, GloVe 같은 워드 임베딩은 “한 단어 = 하나의 고정 벡터”, 즉 고정 임베딩
- 동음이의어 구분 불가 : "배" = 과일/선박/신체 → 항상 같은 벡터
- 다의어(polysemy) 표현 불가 : "paper" = 종이 / 논문 / 과목 → 하나의 벡터에 섞임
- 문맥 변화 반영 불가 : "cold beer" vs "cold weather" → "cold"이 다른 뉘앙스를 가짐

<Contextual Embedding>



Transformer 기반(BERT, GPT 등) Contextual Embedding

- 같은 단어라도 문맥에 따라 다른 벡터를 생성, 보다 정교한 Context을 내포
- He deposited money in the **bank**
- They had a picnic on the river **bank**

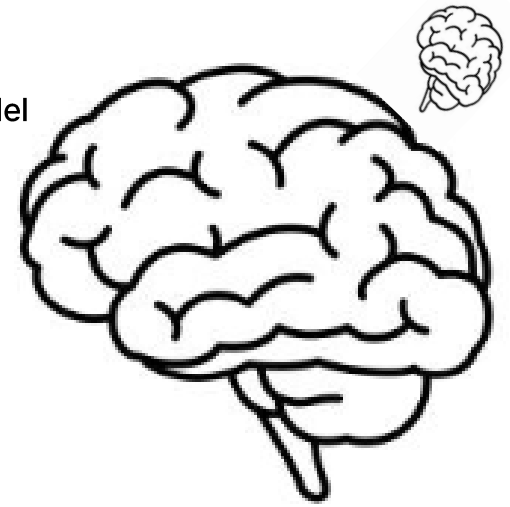
SKALA

미리 배운 걸 잘 써먹자, 전이학습(Transfer Learning)

이걸 잘하면 저것도 잘하지 않을까?

Transfer Learning

- 딥러닝 모델이 스스로 풀기위한 최종 문제를 위해 중간 단계의 개념들을 구조화 하는 것 = Representation Learning
- 풀어야 할 문제가 유사하다면 미리 중간 단계의 개념들을 구조화 해 두어도 되지 않을까?
- 이렇게 유사한 데이터로 미리 학습된 모델을 Pretrained Model이라 함
- 사전에 학습된 Pretrained Model을 바탕으로 나의 최종 문제를 풀어내는 접근 = Transfer Learning
- 초기에 이미지(Vision) 분야를 바탕으로 발전
- 자연어 영역에서 활용할 수 있는 대표적인 Pretrained Model은 Word Embedding과 Language Model



SKALA

대규모 데이터와 Transformer의 만남, GPT

대규모 데이터와 엄청나게 큰 Transformer 모델이 만난다면 어떤 일이 벌어질까?

GPT(Generative Pretrained Transformer).. Better, Faster, Stronger

- Transformer는 기존 모델과는 다르게 분산 컴퓨팅(distributed computing)을 가능하게 한다는 점에서 큰 의미가 있는데, GPT-2,3 같이 미친듯이 큰 모델이 나오게 된 가장 중요한 이유
- GPT가 다른 모델보다 월등하게 좋은 성능을 보이는 가장 큰 이유는 학습 데이터의 양

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. "Weight in training mix" refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

- 약 500bil (5,000억) Token이 포함된 데이터셋 학습
- Common Crawl : 웹페이지의 텍스트를 크롤링 방식을 통해 수집한 자료
- 45TB

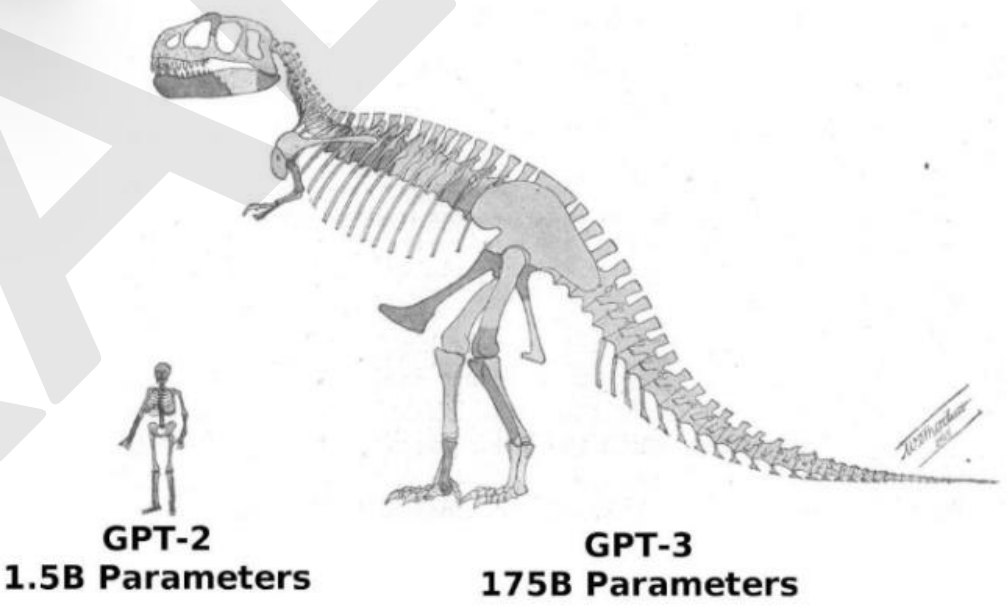
SKALA

Scale up, Scale up.. LLM 시대..

대규모 데이터와 엄청나게 큰 Transformer 모델이 만난다면 어떤 일이 벌어질까?

<u>세 버전의 가장 큰 차이는 Size</u>

- GPT1 - 2018, Improving Language Understanding with Unsupervised Learning
- GPT2 - 2019, Language Models are Unsupervised Multitask Learners
- GPT3 - 2020, Language Models are Few-shot Learners
- GPT3의 Parameter수는 GPT2의 100배+



SKALA

LLM과 Transformer 아키텍처

Math Essentials For Understanding Transformers

- Distance
- Inner product
- linear transformation

본 문서는 SK㈜AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

서로 닮았다는 것

서로 닮았다는 것을 어떻게 측정할 것인가?

Similarity

- 두 객체는 얼마나 닮았는가? 유사한가? .. how much alike
- 0~1 사이의 값을 가짐, 서로 비슷할수록 값이 큼
- 0 : No Similarity, 1: Completely Similar

Dissimilarity (=Distance)

- 두 객체는 얼마나 닮지 않았는가? 떨어져 있는가? ..how much different
- 최소값은 0, 서로 비슷하지 않을수록 값이 큼
- 0 : No Distance

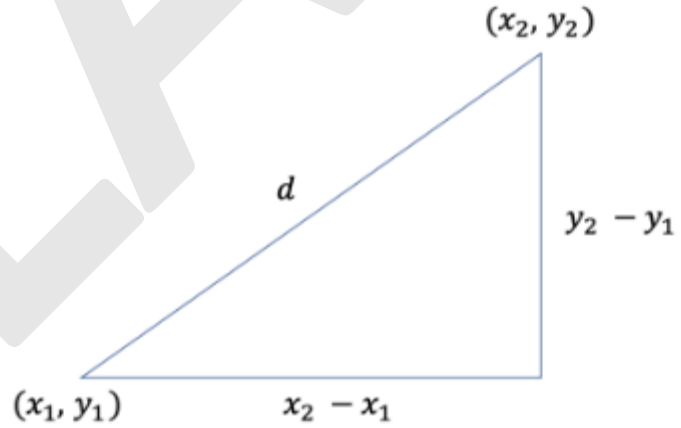
Similarity vs Distance

- 유사도(Similarity)와 거리(Distance)는 닮음의 서로 다른 표현
- **1 – Similarity = Distance**
- 두 객체가 Completely Similar(1) 하다면, 두 객체 간의 거리는 1-1=0

SKALA

Euclidean Distance

두 데이터 간의 직선 거리



Concept

- 직관적이고 구현이 간단하기 때문에 가장 많이 사용되는 척도의 하나
- 벡터의 크기를 측정하는 것이 중요한 경우 효과적
- 각 차원의 차를 제곱해서 모두 더한 값의 제곱근, 피타고라스 정리

$$d(i,j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \cdots + (x_{in} - x_{jn})^2}$$

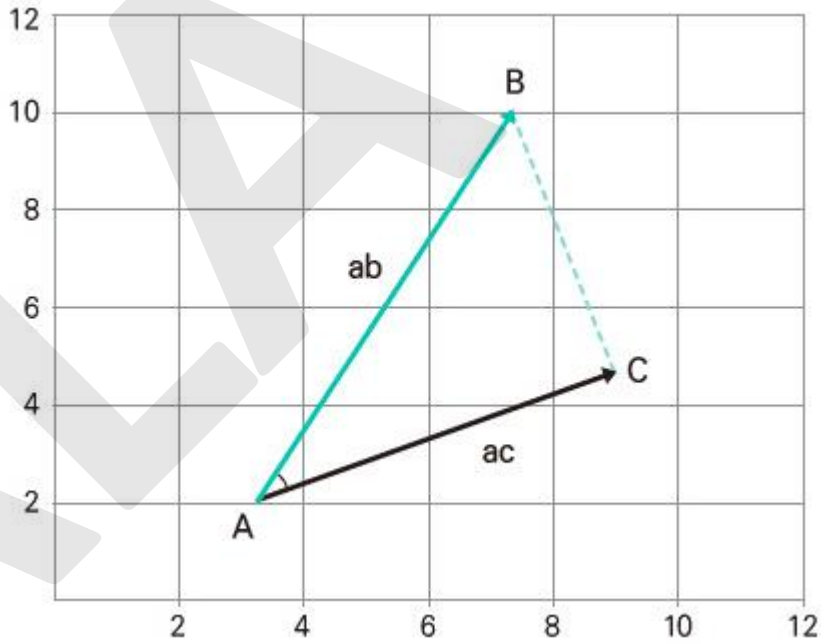
Example

- 세개의 객체가 있을 때, X1과 X2, X1과 X3의 거리를 구해보자
- X1(1,2,2) X2(2,3,1) X3(1,2,4)
- Distance(X1, X2) = sqrt{ $(1-2)^2 + (2-3)^2 + (2-1)^2$ } = sqrt(1 + 1 + 1) = 1.732..
- Distance(X1, X3) = sqrt{ $(1-1)^2 + (2-2)^2 + (2-4)^2$ } = sqrt(0 + 0 + 4) = 2

SKALA

Cosine Similarity

각도 기반의 유사도 측정



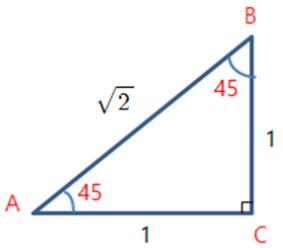
<u>산출 원리</u>

- 코사인(cosine) 삼각함수를 이용
- 코사인은 2차원 평면에서 볼 때 한 지점에서 출발한 방향을 가진 두 값(벡터)의 길이에 대한 비율을 의미
- Ex) 출발점 A로부터 B까지의 거리를 ab, A에서 C까지의 거리를 ac라고 할 때

  → Cosine A = $\frac{ac}{ab}$, 밑변길이 / 빗변길이

- 벡터AB와 벡터AC의 방향이 같다면? → A의 각도는 0, Cosine A = 1
- A의 각도가 90도 라면?

  → Cosine A = 0 , 빗변과 높이는 한없이 커지고..밑변은 작아지고..
- 벡터AB와 벡터AC의 방향이 정반대라면? → A의 각도는 180도, Cosine A = -1

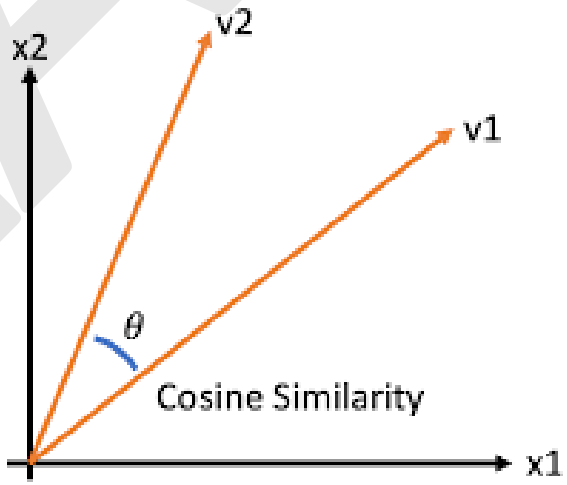




SKALA

Cosine Similarity

각도 기반의 유사도 측정



Concept

- 두 벡터가 가리키는 방향이 얼마나 유사한지 측정 (벡터 크기 무관)
- 방향이 완전히 일치하면 1, 완전히 반대이면 -1
- 벡터의 모든 원소가 양수라면, 코사인 유사도의 최소값은 0
- 1 - Cosine Similarity = Cosine Distance

- 두 값의 방향성이 정확히 일치하는 0도일 때 값이 1이므로 가장 유사도가 높고 각도가 늘어날수록 1보다 값이 작아지며 직교할 때(직각일때)이 유사도가 가장 낮다고 판단
- 유사도는 0과 1 사이의 범위를 가지므로 코사인 유사도는 보통 두 값의 방향이 0도~90도일 때를 기준으로 측정

SKALA

Cosine Similarity

각도 기반의 유사도 측정

Calculation

- 내적의 결과를 총 벡터 크기로 정규화(L2 Normalization)

$$similarity = cos(\Theta) = \frac{A \cdot B}{||A||\,||B||} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n}(A_i)^2} \times \sqrt{\sum_{i=1}^{n}(B_i)^2}}$$

Why use?

- 유사도 기준으로 거리를 사용한다면? → 벡터의 방향과 크기를 고려하는 것
- 유사도 기준으로 코사인 값을 사용한다면? → 벡터의 방향만을 고려하는 것
- 코사인 유사도 → 문서의 길이가 다르기 때문에 벡터간 거리를 벡터의 크기로 정규화 하여 사용

Example - A(1,1,0), C(3,3,0)의 코사인 유사도

- 분자 : (1*3)+(1*3)+(0*0) = 6
- 분모 : sqrt(1^2+1^2+0^2) * sqrt(3^2+3^2+0^2)

  = 1.414.. * 4.242.. = 6

→ AC의 코사인 유사도 = 6/6 = 1

SKALA

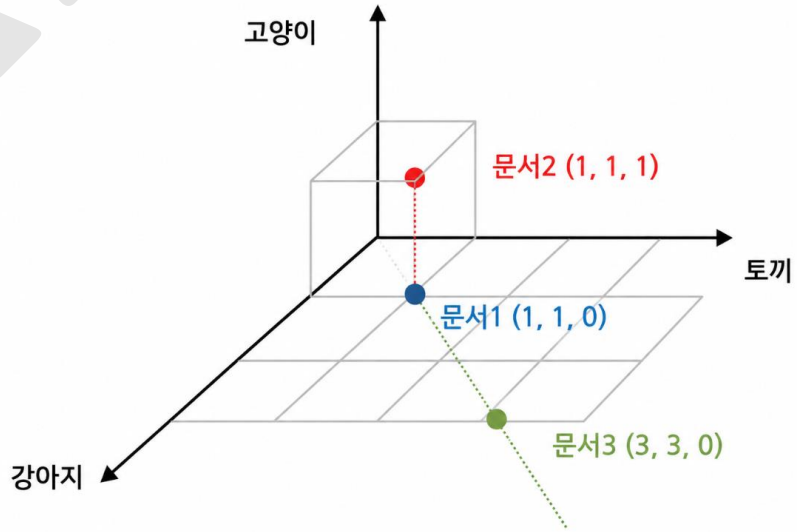
Case Study

자연어 처리에서의 유사도 측정

문서1은 문서2와 문서3중 어느 문서와 더 유사한가요?

- 유클리드 거리와 코사인 유사도를 산출하여 비교해보고 그 의미를 파악해보세요

	내용	Vector(강아지,고양이,토끼)
문서1	강아지 고양이	(1,1,0)
문서2	강아지 고양이 토끼	(1,1,1)
문서3	강아지 강아지 강아지 고양이 고양이 고양이	(3,3,0)



SKALA

Dot Product

내적의 의미

<u>Calculation</u>

- 두 벡터 a, b가 이루는 각이 θ일 때 다음을 두 벡터 a,b의 내적이라 함

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta = \sum_{i=1}^{d} a_i b_i$$

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\mathbf{a}^T\mathbf{b} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

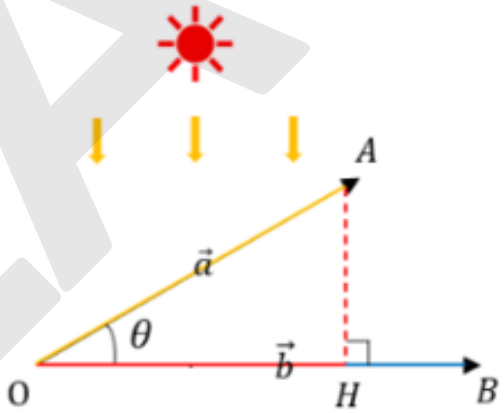
$$= 1 \times 4 + 2 \times 5 + 3 \times 6$$

$$= 4 + 10 + 18 = 32$$

SKALA

Dot Product

내적의 의미



Implicit meaning

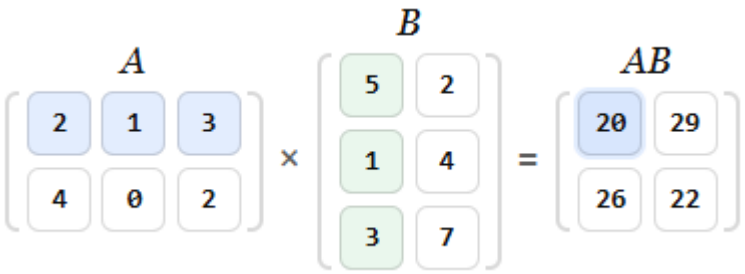
- 벡터의 내적은 기하학적으로 한 벡터를 다른 벡터 위로 정사영시킨 길이와 그 다른 벡터의 길이를 곱한 것을 의미
- 크기와 방향을 모두 고려한 유사도 개념
- 정사영된 길이가 짧다는 것 = 벡터간 각도가 크다 = 유사도가 작다
- 정사영된 길이가 길다는 것 = 벡터간 각도가 작다 = 유사도가 크다

$$\vec{a} \cdot \vec{b} = |\vec{a}| \times |\vec{b}| \times cos\,\theta$$
$$= |\vec{b}| \times |\vec{a}| \times cos\,\theta$$
$$= \overline{OH} \times \overline{OB}$$

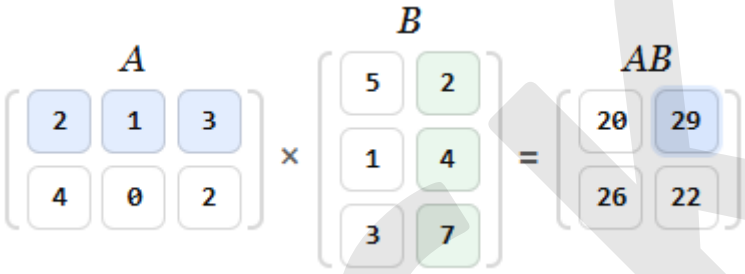
SKALA

Matrix Multiplication

A(2×3) B(3×2) AB(2×2)



$(AB)_{11}=(2\cdot 5)+(1\cdot 1)+(3\cdot 3)=20$



$(AB)_{12}=(2\cdot 2)+(1\cdot 4)+(3\cdot 7)=29$

✓ 행렬곱(Matrix Multiplication)의 각 원소는 행(Row) 벡터와 열(Column) 벡터의 내적(Dot Product)

✓ Matrix Multiplication = Row × Column Dot Products

SKALA

선형변환, linear transformation

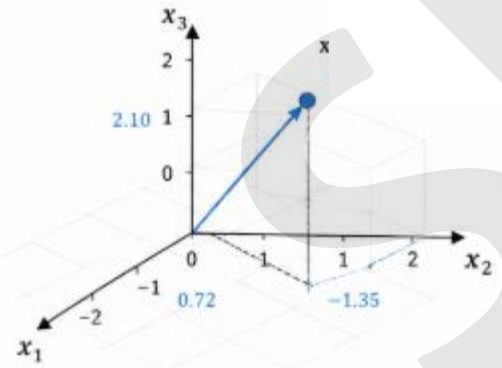
$$\mathbf{x} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \quad W = \begin{bmatrix} 2 & 0 \\ 1 & -1 \\ 0 & 1 \end{bmatrix} \quad = \mathbf{y} = \begin{bmatrix} 2 & 1 \end{bmatrix}$$

- 3차원 벡터 x는 행렬 W와 곱을 통해 2차원 벡터로 선형변환 됨

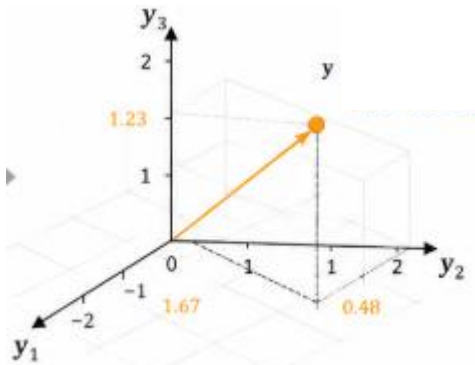
Definition

- 선형변환은 벡터의 덧셈과 배수 관계를 유지하면서, 벡터를 다른 벡터로 매핑하는 함수, 행렬곱은 이러한 선형변환을 계산하는 방법
- 벡터를 회전·확대·축소·반사하거나 다른 차원의 벡터로 바꾸되, 공간의 직선성과 원점을 보존하는 변환

$$\mathbf{x} = \underset{(1\times3)}{\begin{bmatrix} 0.72 & -1.35 & 2.10 \end{bmatrix}} \quad W = \underset{(3\times3)}{\begin{bmatrix} 0.80 & -0.40 & 0.30 \\ -0.50 & 0.90 & -0.10 \\ 0.30 & 0.20 & 0.70 \end{bmatrix}} \quad = \quad \mathbf{y} = \underset{(1\times3)}{\begin{bmatrix} 1.67 & 0.48 & 1.23 \end{bmatrix}}$$

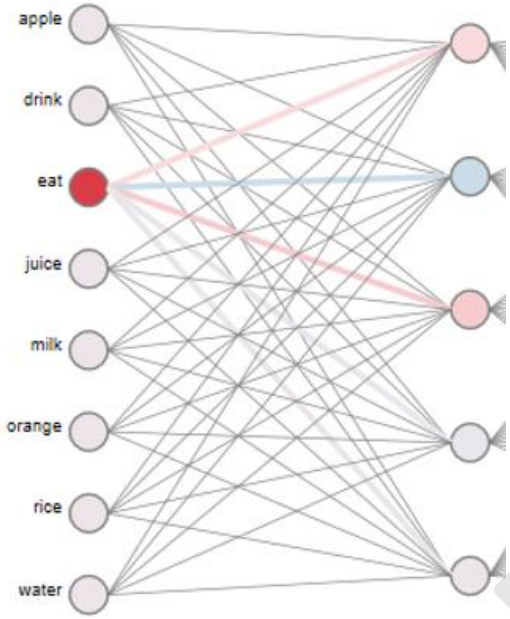


- 선형변환은 벡터 x를 새로운 좌표로 이동시킴



SKALA

선형변환, linear transformation



X (8×10)

	d_1	d_2	d_3	⋯	d_{10}
apple	x_{11}	x_{12}	x_{13}	⋯	x_{110}
drink	x_{21}	x_{22}	x_{23}	⋯	x_{210}
eat	x_{31}	x_{32}	x_{33}	⋯	x_{310}
juice	x_{41}	x_{42}	x_{43}	⋯	x_{410}
milk	x_{51}	x_{52}	x_{53}	⋯	x_{510}
orange	x_{61}	x_{62}	x_{63}	⋯	x_{610}
rice	x_{71}	x_{72}	x_{73}	⋯	x_{710}
water	x_{81}	x_{82}	x_{83}	⋯	x_{810}

×

W (10×5)

	h_1	h_2	h_3	h_4	h_5
d_1	w_{11}	w_{12}	w_{13}	w_{14}	w_{15}
d_2	w_{21}	w_{22}	w_{23}	w_{24}	w_{25}
d_3	w_{31}	w_{32}	w_{33}	w_{34}	w_{35}
⋮	⋮	⋮	⋮	⋮	⋮
d_{10}	w_{101}	w_{102}	w_{103}	w_{104}	w_{105}

=

Y (8×5)

	h_1	h_2	h_3	h_4	h_5
apple	y_{11}	y_{12}	y_{13}	y_{14}	y_{15}
drink	y_{21}	y_{22}	y_{23}	y_{24}	y_{25}
eat	y_{31}	y_{32}	y_{33}	y_{34}	y_{35}
juice	y_{41}	y_{42}	y_{43}	y_{44}	y_{45}
milk	y_{51}	y_{52}	y_{53}	y_{54}	y_{55}
orange	y_{61}	y_{62}	y_{63}	y_{64}	y_{65}
rice	y_{71}	y_{72}	y_{73}	y_{74}	y_{75}
water	y_{81}	y_{82}	y_{83}	y_{84}	y_{85}

X (8×10) × **W (10×5)** = **Y (8×5)**

8개 단어 10차원 벡터 | 10차원 → 5차원 선형변환 | 8개 단어 5차원 벡터

<u>Weight Matrix 를 행렬곱 하는 의미</u>

- Apple = [0.34.., 0.12.., 0.56.., 0.78.., 0.39.., 0.61.., 0.28.., 0.92.., 0.24.., 0.38..] 10차원 벡터
- 딥러닝의 학습이란, Weight Matrix를 조금씩 수정하여 데이터가 '의미 있는 위치'에 놓이도록 벡터 공간을 계속 재배치하는 과정
- 단어가 변하는 것이 아니라, 단어를 표현하는 벡터가 변하는 것

SKALA

Softmax function

Softmax: Scores to Probabilities

Concept

- Softmax는 여러 실수 점수를 비교 가능한 확률 분포로 변환
- 각 점수가 전체에서 차지하는 상대적 비중을 계산
- 모든 출력값은 0과 1 사이
- 출력값의 합은 1
- 입력값의 상대적인 크기 관계를 유지



$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

SKALA

LLM과 Transformer 아키텍처

Transformer

- Attention Is All You Need
- Self Attention
- Multi-head Attention
- Masked Self Attention

본 문서는 SK㈜AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Attention Is All You Need

현대 Transformer 기반 모델(BERT, GPT 등)의 출발점

Motivation

- 2017년 Google Brain 팀에서 “Neural Machine Translation(NMT)” 프로젝트를 하던 연구자들에 의해 작성
- 당시 주류 기계번역 모델은 한계를 가지고 있었음
  - RNN/LSTM/GRU 기반 Seq2Seq 모델 : 병렬화 어려움, 장기의존성(long-term dependency) 문제
  - CNN 기반 모델 : 멀리 떨어진 토큰간 관계위해 너무 깊은 계층 필요

Proposal

- Transformer Architecture
- RNN이나 CNN 없이 Self-Attention만으로 시퀀스를 처리하자

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani[*] Google Brain avaswani@google.com
Noam Shazeer[*] Google Brain noam@google.com
Niki Parmar[*] Google Research nikip@google.com
Jakob Uszkoreit[*] Google Research usz@google.com
Llion Jones[*] Google Research llion@google.com
Aidan N. Gomez[*] [†] University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser[*] Google Brain lukaszkaiser@google.com
Illia Polosukhin[*] [‡] illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

[*] Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†] Work performed while at Google Brain.

[‡] Work performed while at Google Research.

SKALA

Attention Is All You Need

현대 Transformer 기반 모델(BERT, GPT 등)의 출발점

Results by the paper

- WMT (Workshop on Machine Translation)는 기계번역 분야의 가장 권위있는 학회로, 매년 번역경진대회를 개최
- 연구자들은 공통된 데이터셋으로 모델 성능을 비교할 수 있음
- WMT 2014 English–German와 English–French 데이터를 통해 성과 측정

- English–German → BLEU 28.4 (당시 SOTA)
- English–French → BLEU 41.8 (당시 SOTA)
- 기존의 모델들과 비교해 훨씬 적은 연산 비용(training cost)으로 SOTA 달성

[참고] BLEU : Bilingual Evaluation Understudy

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

SKALA

Attention Is All You Need

현대 Transformer 기반 모델(BERT, GPT 등)의 출발점

Contribution

- Self-Attention만으로 이루어진 최초의 Seq2Seq 모델제안
- RNN/CNN 의존 없이 병렬 학습 + 장기 의존성 처리 가능함을 증명
- 이후 모든 대규모 언어모델(BERT, GPT 등)의 기반 아키텍처가 됨

SKALA

Attention Is All You Need

현대 Transformer 기반 모델(BERT, GPT 등)의 출발점

Transformer Architecture

- 인코더-디코더 구조

Encoder

입력 문장을 여러 층의 **self-attention**

+ feed-forward network로 **벡터로**

추출

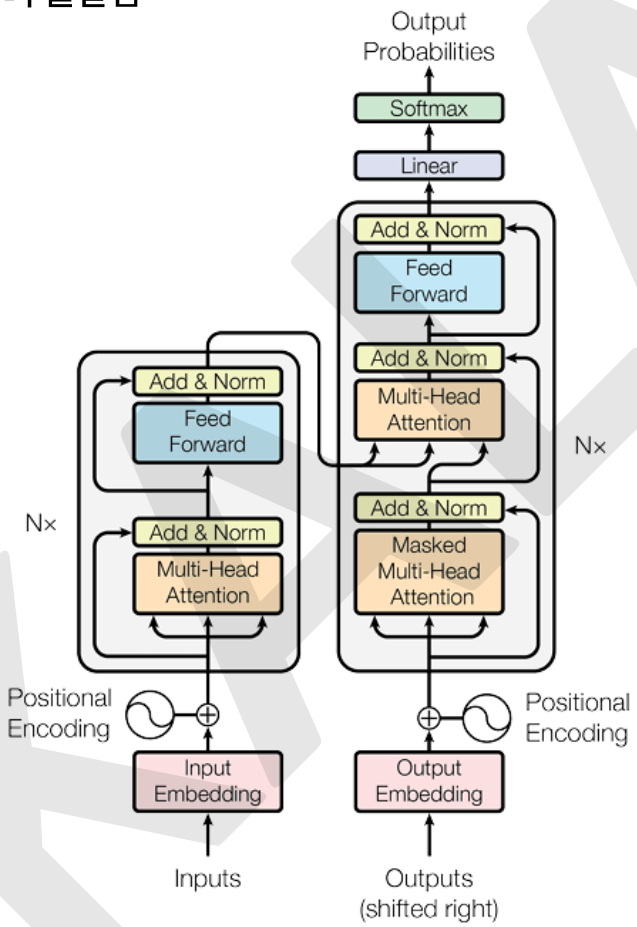


Figure 1: The Transformer - model architecture.

Decoder

인코딩된 벡터와 자기 자신의

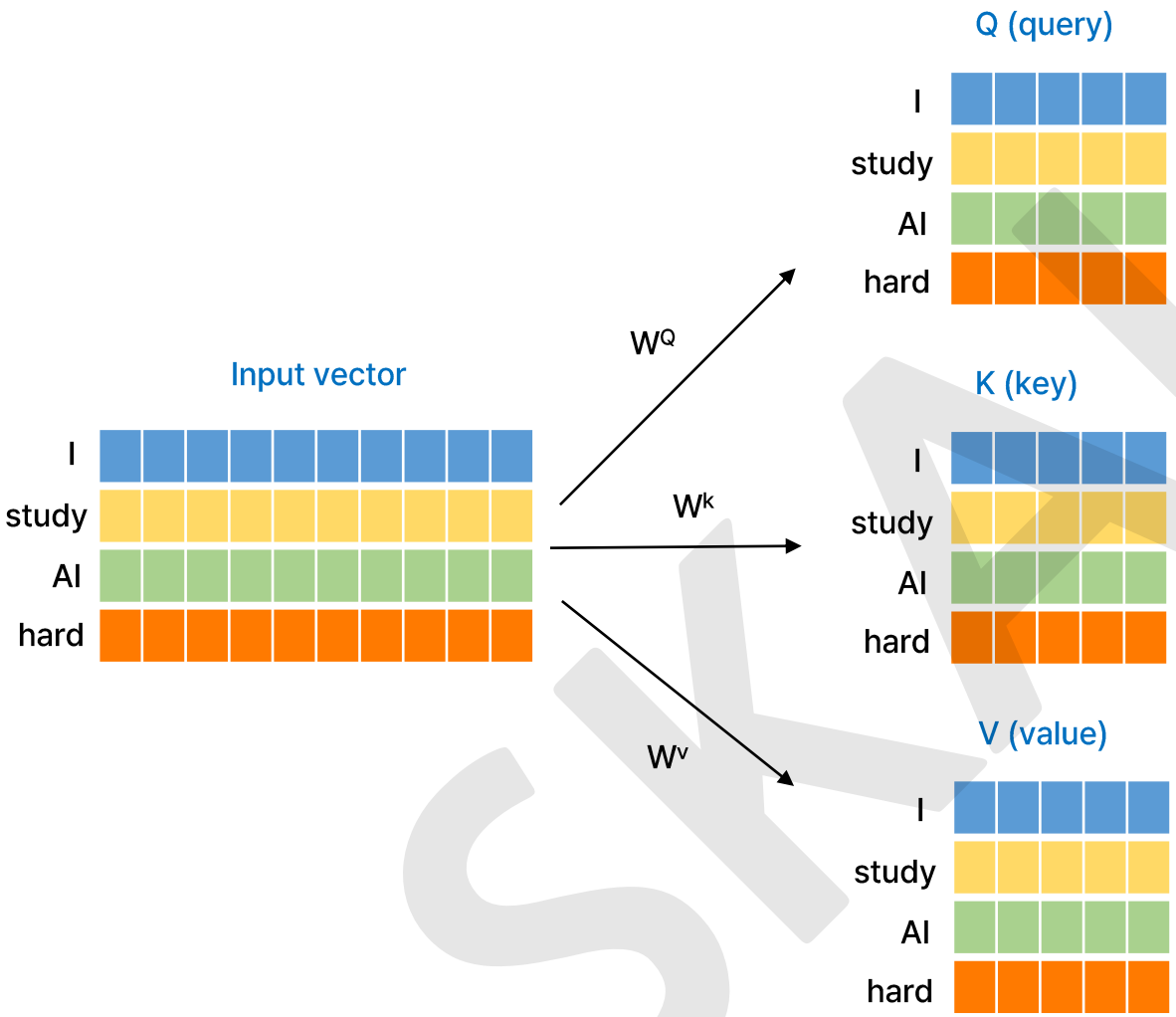
출력에 대한 masked self-attention을

통해 **번역(목표 문장)을 생성**

SKALA

Transformer Encoder

Scaled Dot-Product Attention – Self Attention

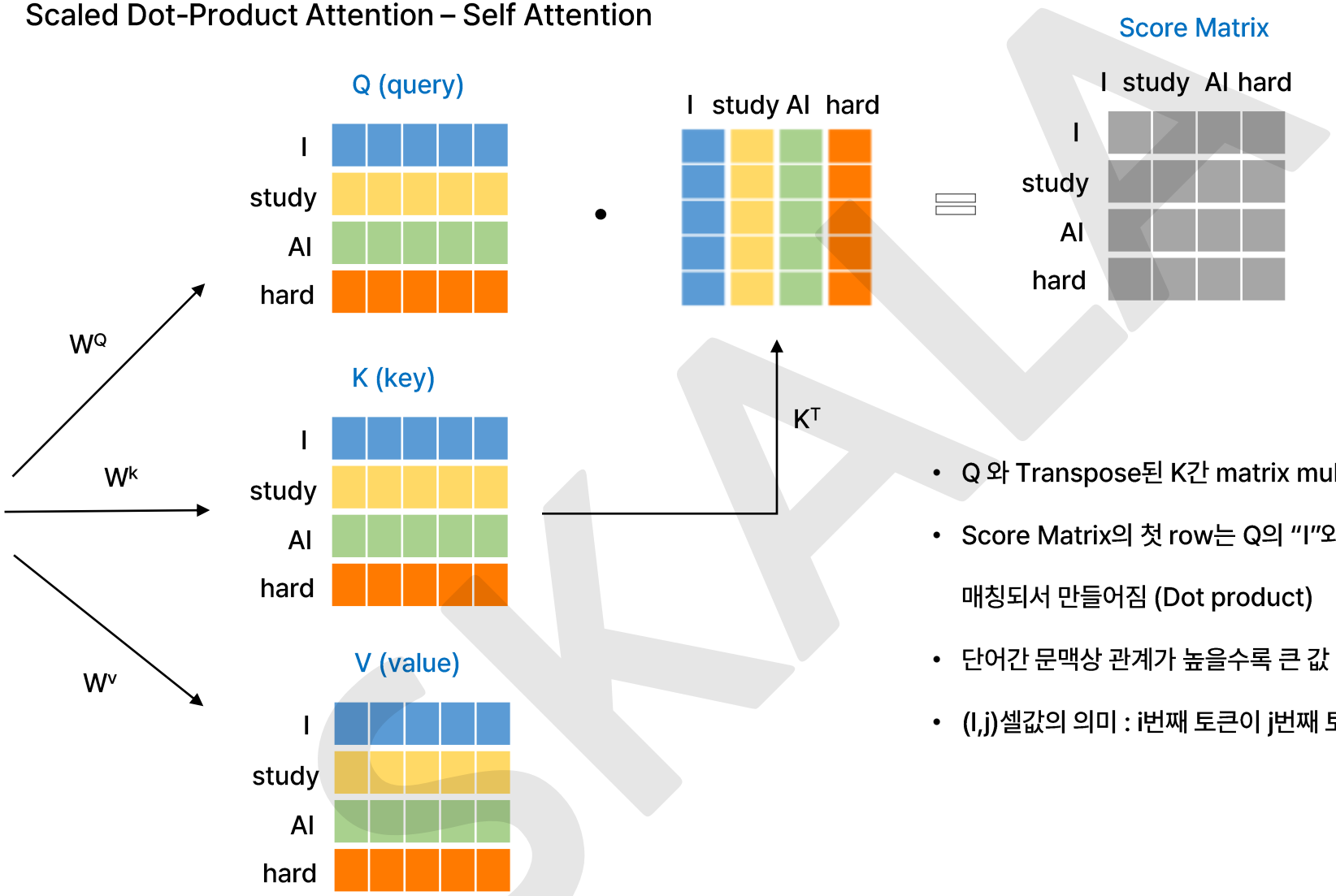


- 내가 누구를 참고하지? 어디에 집중할래?
- Query 토큰들로부터 참고될 수 있는 정보
- 실제 전달되는 정보

SKALA

Transformer Encoder

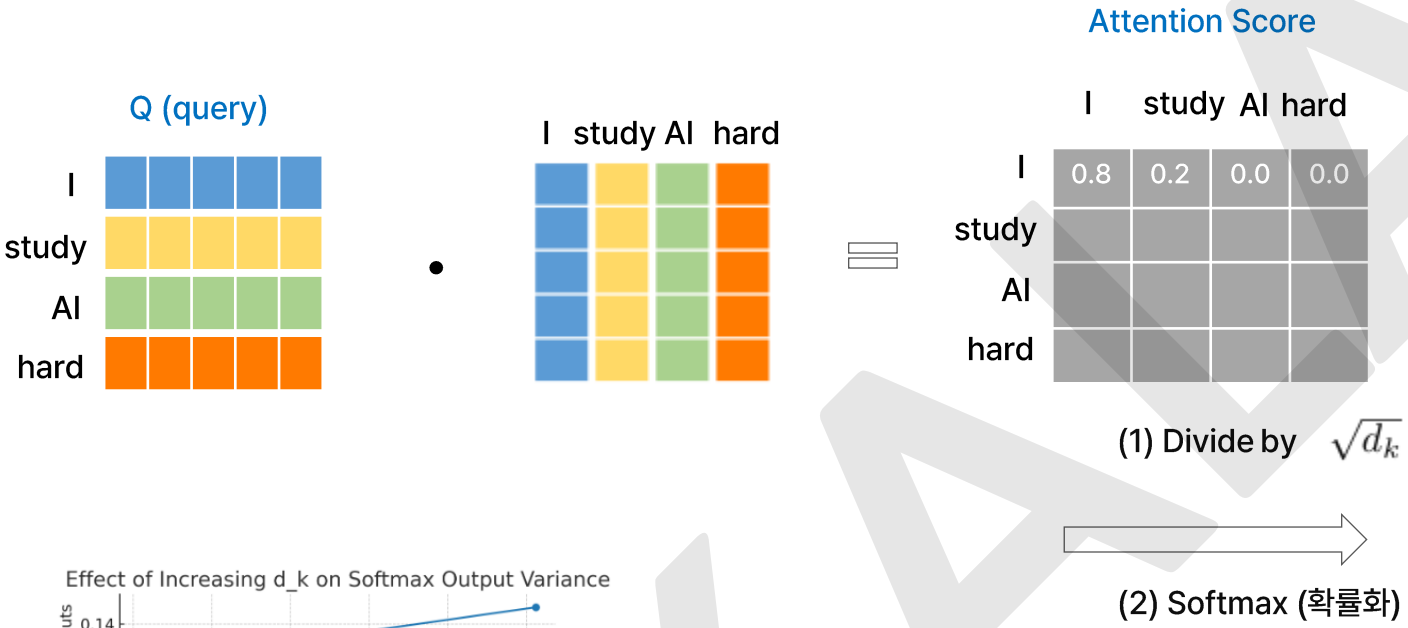
Scaled Dot-Product Attention – Self Attention

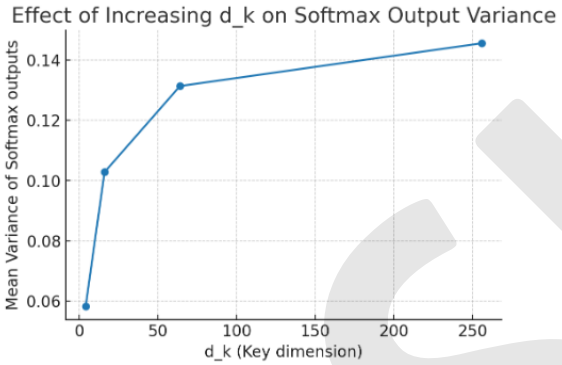


- Q 와 Transpose된 K간 matrix multiplication
- Score Matrix의 첫 row는 Q의 "I"와 K의 각 토큰 벡터가 매칭되서 만들어짐 (Dot product)
- 단어간 문맥상 관계가 높을수록 큰 값
- (I,j)셀값의 의미 : i번째 토큰이 j번째 토큰을 얼마나 주목하느냐?

Transformer Encoder

Scaled Dot-Product Attention – Self Attention



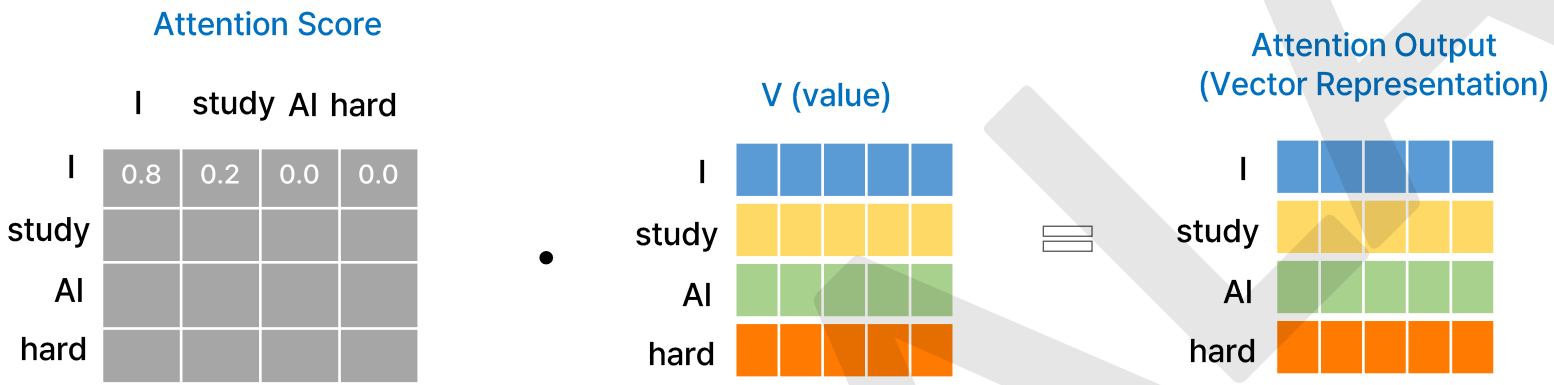


- Q, K 벡터 차원(d_k)이 커질수록 내적 값의 분산이 커져서, Softmax 함수의 기울기(gradient)가 매우 작아지는 문제 해소하기 위해 K 차원수의 제곱근으로 나누어줌, 일종의 Scaling
- 이후 Softmax 를 통해 확률화를 진행
- 이로써 내적 값의 크기 범위가 안정화되고, Softmax가 적당한 기울기를 유지

SKALA

Transformer Encoder

Scaled Dot-Product Attention – Self Attention

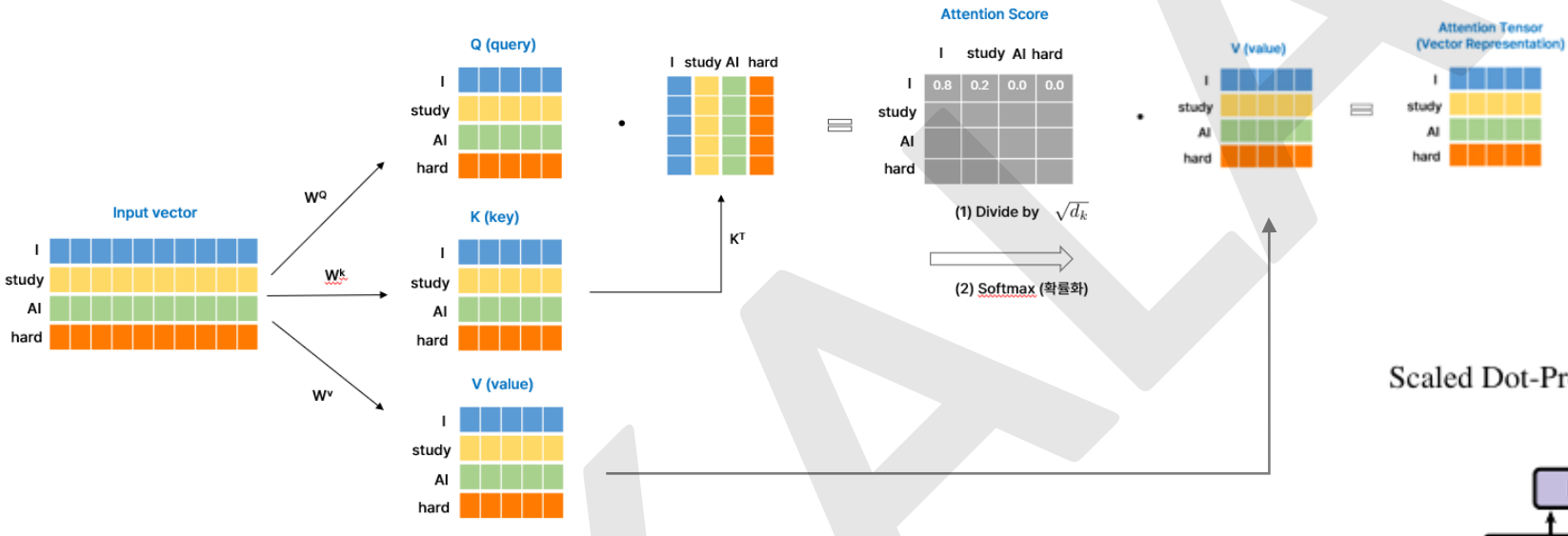


- 토큰간 연관성(유사도)를 담고 있는 Attention Score Matrix는 각 토큰이 다른 토큰을 얼마나 주목해야 하는지에 대한 가중치 행렬
- 이에 실제 정보(V)를 행렬곱하여 문맥 벡터를 생성 = 각 토큰이 다른 토큰들의 정보를 비율대로 섞어 만든 새 표현

SKALA

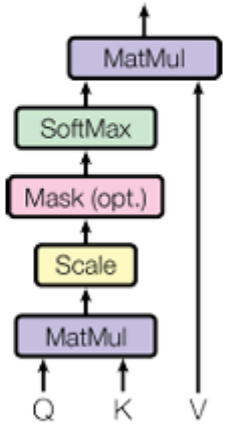
Transformer Encoder

Scaled Dot-Product Attention – Self Attention



$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

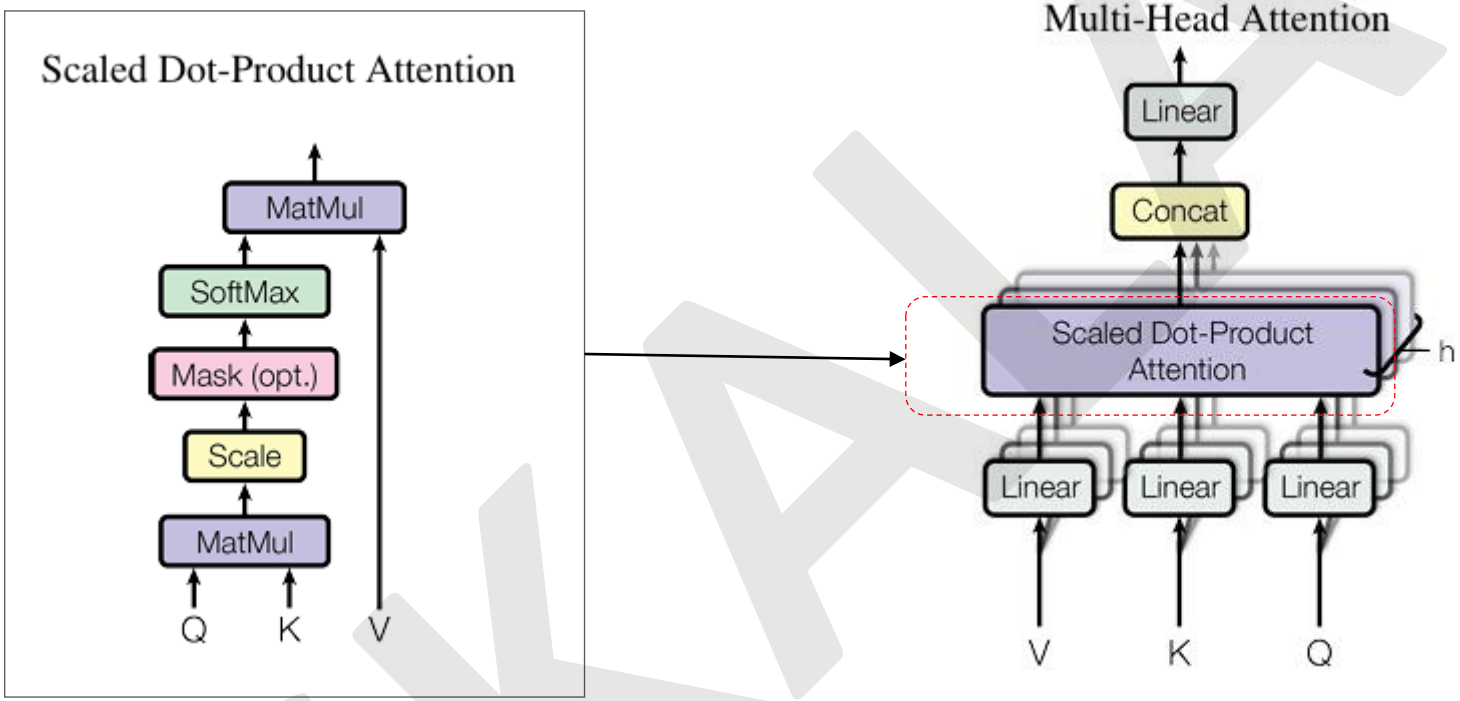
Scaled Dot-Product Attention



SKALA

Transformer Encoder

Multi-head Attention

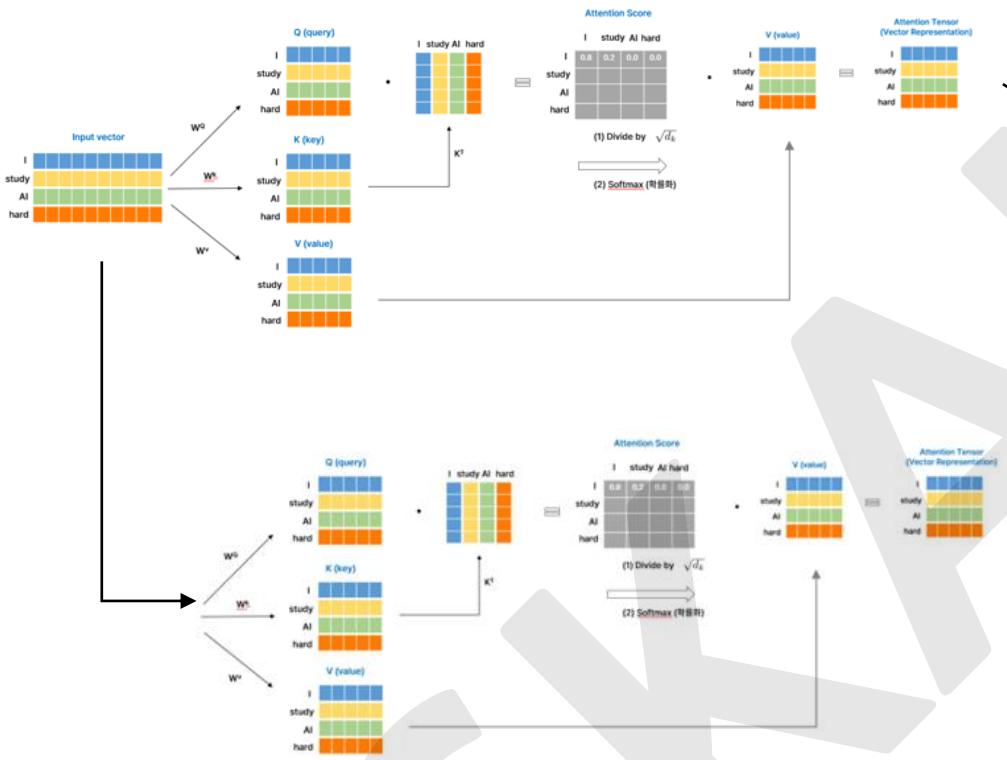


- 입력 시퀀스를 서로 다른 관점에서 주목(attend) 해보자
- 여러 "관점"을 합쳐서 다시 하나의 통합 표현으로 만들자

SKALA

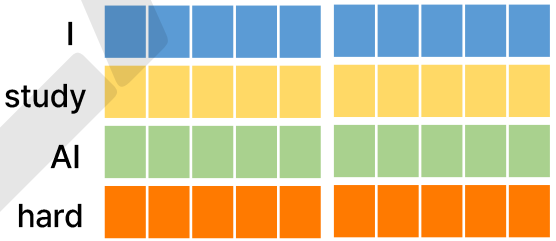
Transformer Encoder

Multi-head Attention



Linear transformation

Attention Output Concatenate



Why Linear Layer?

- Concatenate한 벡터는 헤드별 특징이 흩어져 있음
- Ex) 문법적관계 집중(첫번째 output) + 의미적 유사성 집중(두번째 output)
- 이를 융합(재조합)해서 모델이 사용하기 적절한 표현 공간으로 투영
- 다양한 관점의 정보를 통합하고, 차원을 맞추며, 다음 레이어가 쓸 수 있는 형태로 재배치하는 과정

SKALA

Transformer Encoder

Encoder Process

Normalization

- LayerNorm은 각 토큰의 벡터 차원별 평균과 분산을 정규화해서, 입력 분포가 일정하게 유지되도록 도움
- 그 결과 학습이 빠르고 안정적이 됨, 그래디언트 소실 방지

잔차 연결, Residual Connection

- 정보 보존 : 어텐션이 계산한 값만 쓰면, 원래 입력 정보가 손실될 수 있음
- → 기존 정보 위에 어탠션이 계산한 추가적인 관계 정보를 보강하는 방식
- 학습안정화 : 입력을 그대로 출력에 더해줌으로써, Gradient 소실 문제를 방지함. 결과적으로 학습이 더 잘 수렴

$$\text{Output} = \text{LayerNorm}(\text{Input} + \text{MultiHeadAttention}(\text{Input}))$$

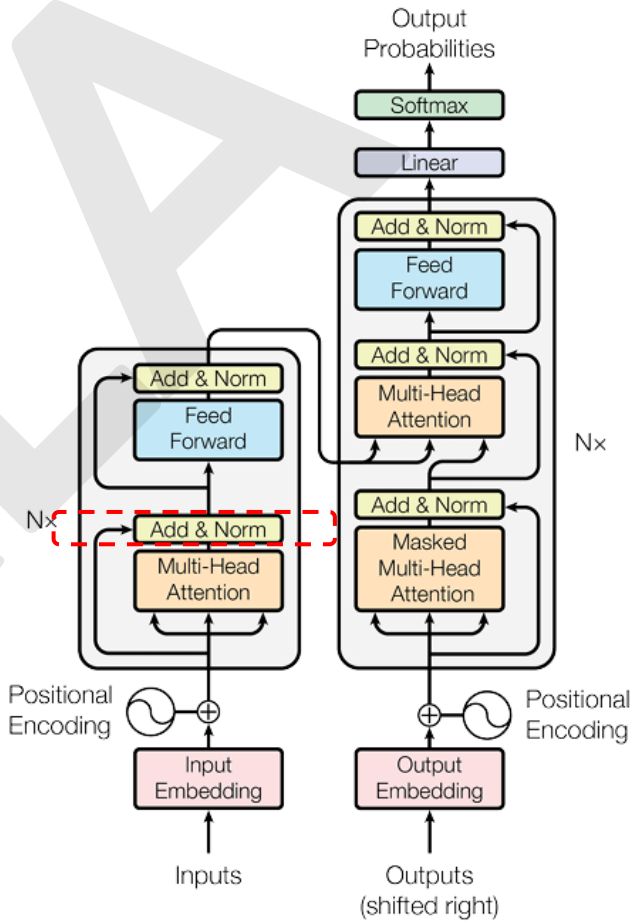


Figure 1: The Transformer - model architecture.

SKALA

Transformer Encoder

Encoder Process

Feed Forward

- Attention은 거의 선형변환의 조합이므로, 모델 표현력이 부족할 수 있음
- 문맥이 반영된 벡터표현을 더 풍부하게 하기 위해 비선형성 추가 (ReLU)

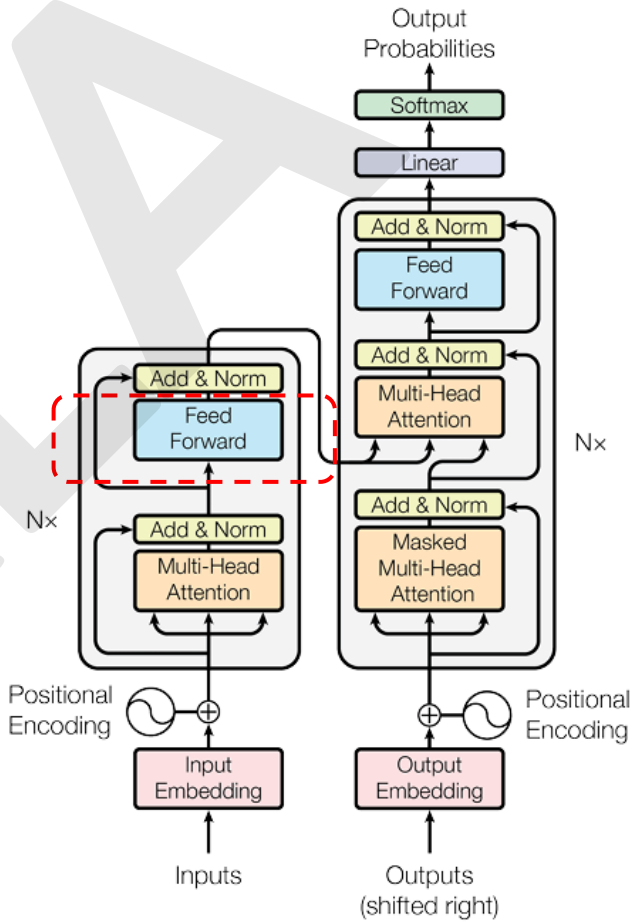


Figure 1: The Transformer - model architecture.

SKALA

Transformer Encoder

Positional Encoding

Problem

- 트랜스포머는 '모든 토큰을 함께 보는 컨셉' → 토큰간 순서를 직접 알 수 없음
- Ex) 나는 밥을 먹었다 vs 밥이 나를 먹었다
- Query, Key, Value로 이루어진 어텐션 구조 자체가 토큰의 순서와는 상관없이 모두 연결되어 있어 추가적 정보없이 두 문장의 차이구분이 어려움

Solution

- 포지션 임베딩 레이어 학습
- 학습없이 삼각함수를 활용, 정해진 계산식에 따라 결과값을 인코딩값으로 사용

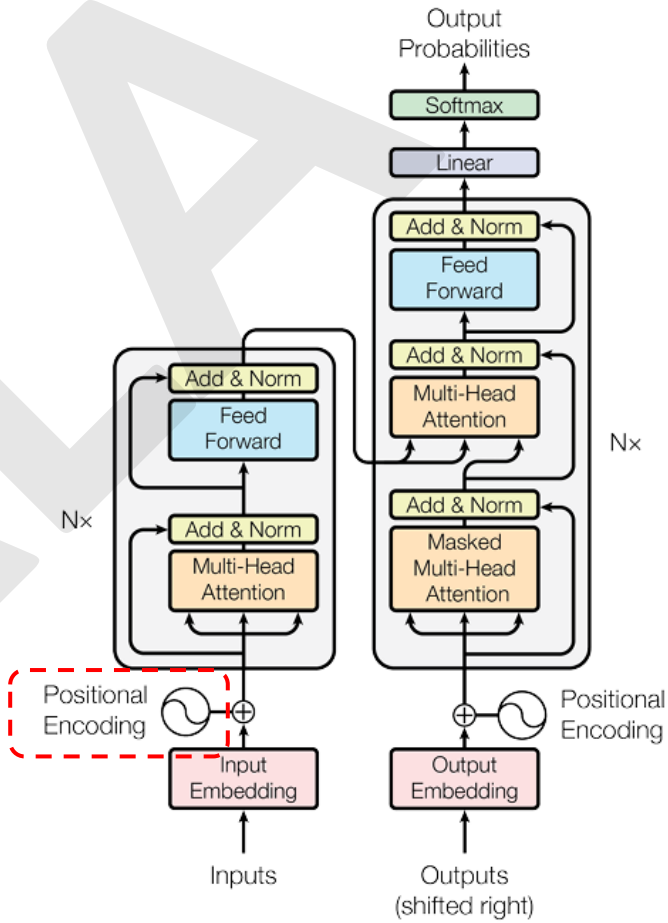


Figure 1: The Transformer - model architecture.

SKALA

Transformer Decoder

Concept

Problem

- 어탠션 메카니즘은 모든 단어들이 나와 있을 때 서로 간의 문맥을 파악하는 것이지 그 주어진 문장 그 다음에 토큰을 예상하기 위한 구조는 아님
- 주어진 정보를 가지고 다음 단어를 예측하는 구조로 변경하려면..

Ideation, 셀프어탠션 디코더로 사용하기

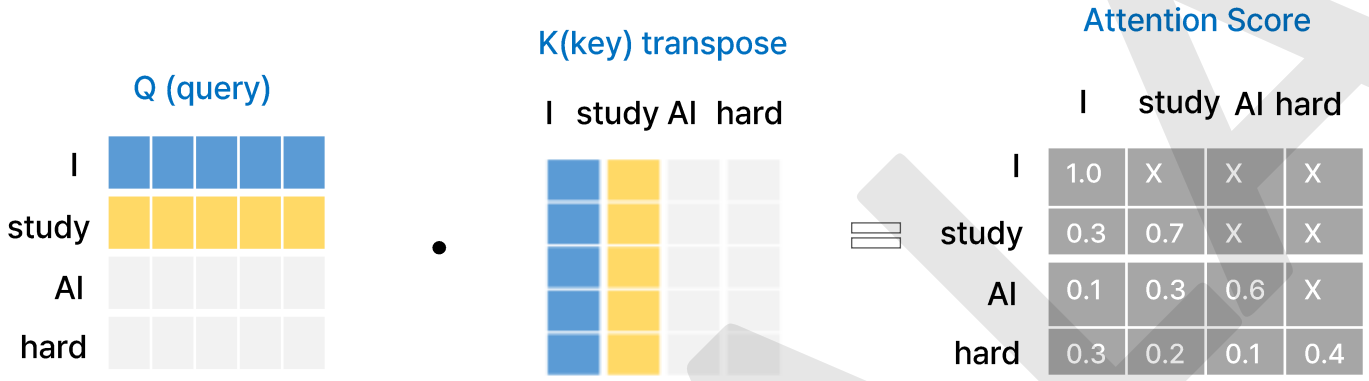
- I _____
- I study _____
- I study AI _____

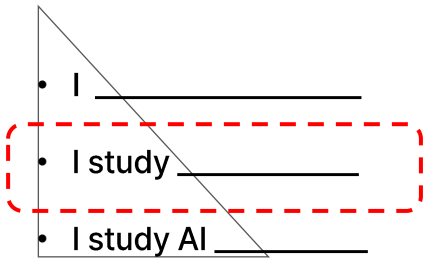
- 주어진 토큰들만 가지고 서로를 참조하여 알맞은 피처들을 뽑아내고 그를 기반으로 다음 토큰을 예측하도록 한다면..
- 생성은 주어진 토큰안에서 다음 토큰을 예상하는 것, 그렇게 해보자

SKALA

Transformer Decoder

Masked Self Attention





Learning

- 주어진 토큰들 사이의 문맥을 학습하여 다음 토큰 예측
- 이때 Y는 다음 토큰
- Training step마다 Attention Score와 Ground Truth 와 비교하여 Loss 계산

SKALA

수고하셨습니다.

본 문서는 SK㈜AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.